

Panima

(פנים-ה)

שם תלמיד: דניאל בנטוב
ת"ז: 209815158
שם המנחה: עידן פלדברג
תאריך: 25.05.25
שם הבית ספר: תיכון אלון

Table of Contents

1	Panima
1	(פנים-ה)
4	מבוא
4	ייזום:
4	תיאור ראשוני של המערכת, אתגרים, תועלות וחסכוניות, הגדרת יעדים ומטרות
5	סקירת פתרונות קיימים
5	תיחום וסקירת טכנולוגיות הפרויקט
6	פירוט תיאור המערכת:
6	פירוט הבדיקות שמתוכננות לפרויקט
7	ניהול זמנים/תכנון עבודה:
7	ניהול סיכונים והדרכים להתמודד איתם:
8	תיאור תחום הידע – פרק מילולי
9	שם היכולת: כניסה באמצעות זיהוי פנים
9	שם היכולת: שליחת פקודה לשרת
10	שם היכולת: עיבוד בקשת לקוח:
11	שם היכולת: התחברות עם תעודת זהות וסיסמה ועדכון טוקן:
11	מבנה הפרויקט
11	תיאור ארכיטקטורת המערכת:
11	לקוח (CustomTkinter+ client)-
12	שרת (Server.py) :
12	מסד נתונים(Database.py) :
14	שרטוט:
14	תיאור הטכנולוגיות הרלוונטיות:
15	תרשים זרימה Hybrid Encryption:
16	תהליך כניסה באמצעות זיהוי פנים:
17	תהליך כניסה רגילה:
18	תהליך מסך מרכזי:
19	אלגוריתמים:
19	אלגוריתם זיהוי פנים:
19	אלגוריתם הצפנה ושליחת הודעות:
20	תיאור סביבת הפיתוח:
20	פרוטוקול תקשורת:
21	מאגר הודעות:
22	תיאור מסכי המערכת:
30	תרשום זרימה של מסכי המערכת:
31	תיאור מבני הנתונים:
31	פירוט מסד נתונים database:
32	פירוט טבלאות:
34	בנוסף יש טבלה behavior בה אעשה שימוש כאשר אבנה את הלוגיקה לנוכחות.

34	תרשים של הטבלאות:
34	חולשות ואיומים:
34	בשכבת האפליקציה:
35	בשכבת התעבורה:
36	מימוש הפרויקט
36	שם המחלקה Server:
37	שם המחלקה Client:
37	שם המחלקה CustomTkinter:
38	שם המחלקה FaceAnalyzer:
39	שם המחלקה Security:
39	שם המחלקה Database:
40	שם המחלקה NetworkComponent:
42	קטעי קוד מיוחדים:
42	יכולת 1 – זיהוי פנים והשוואה (face_id_login) -
43	יכולת 2 ניהול הודעות בצד השרת מהלקוח (process_
43	message)
44	בדיקות:
46	מדריך למשתמש
46	קבצים:
46	
47	התקנה במערכת:
48	נתונים התחלתיים:
49	רשת:
49	הארכיטקטורה המינימלית הנדרשת להרצת המערכת כוללת:
49	אופן הפעלה המערכת:
54	רפלקציה:
55	ביבליוגרפיה:
55	נסחפים:
56	CLIENT:
62	SERVER
67	Database
84	Security:
85	FaceAnalyzer:
87	CustomTkinter:
109	NetworkComponent:
110	Main:

מבוא

ייזום:

תיאור ראשוני של המערכת, אתגרים, תועלות וחסכוניות, הגדרת יעדים ומטרות

ההשראה שלי לפרויקט הגיעה ממורה לסייבר שמלמד בשכבה שלי (אותי ספציפית לא), תמיד היה לו רעיון לאפליקציה שתסמן נוכחות עם זיהוי פנים, כך שבמקום לבזבז את הכמה דקות הראשונות של שיעור על קריאת שמות, התהליך יתבצע בכמה שניות. חשוב לדעת שככל שהזמן עבר השתנו המטרות שלי לפרויקט, ולמרות שנשאר חלק מהרעיון הכללי הוא כן מאוד שונה ממה שחשבתי לעשות בהתחלה.

כרגע, בזמן הגשת הספר, הפרויקט שלי הוא מערכת שעות של תלמידים ומורים, עם כניסה למערכת באמצעות זיהוי פנים (בנוסף לכניסה רגילה של תעודת זהות וסיסמה). לכך אני מתכנן להוסיף את הרעיון הראשוני של הפרויקט הכולל סימון נוכחות באמצעות זיהוי פנים, בנוסף ללוגיקה כללית של הערות תלמיד (חיסורים, איחורים, הפרעות...)

אתגרים שאני מצפה להם בפרויקט יהיו לגבי לאבטח כניסה בטוחה למשתמשים באמצעות זיהוי פנים, ולמידה על כל הנושא שאדע להסביר מה הולך מאחורי הכלאים. בנוסף לכך לדעתי הבסיס נתונים יהיה קשה לבנות, בגלל כמות הפיצ'רים השונים שאני רוצה לפרויקט.

האפליקציה היא כמובן לתלמידים ומורים, ותאורטית גם להורים של תלמידים שרוצים לראות את מערכת השעות של ילדיהם.

הבעיה שהפרויקט באה לפתור, היא תהליך ארוך של קריאת שמות בתחילת שיעורים, ומערכת משוב שאותו דבר כבר מספר שנים ולא נוחה לשימוש. המערכת תפתור את הבעיות האלה עם אפליקציה יותר קלה לניווט ושימוש, ובנוסף סימון נוכחות יעילה יותר.

סקירת פתרונות קיימים

האפליקציות הדומות לכך הן "משוב" ו"שחף – מערכת שעות", בהן משתמשים תלמידים ברחבי הארץ לצפייה במערכת השעות, אך לכל אחת מהן מספר בעיות.

משוב- משוב היא אפליקציה ותיקה שלא עברה שינויים מהותיים במשך שנים, ולכן קשה לניווט ובעיקר לתהליך ההתחברות. אני יחד עם הרבה תלמידים נכנסים כל פעם עם תעודת זהות וקוד שנשלח למספר טלפון, לפעמים גם מבחן אימון CAPTCHA באמצע. כל התהליך לוקח לפחות 30 שניות והחלק הכי מציק, שאחרי שסוגרים או יוצאים מהאפליקציה (אפילו רק למספר שניות) הוא מוציא אותך החוצה וצריך את כל התהליך שוב.

שחף- "שחף מערכת שעות" משתמש את תלמידי הארץ במערכת השעות שלהם, מכיוון שאין דרך לעדכן במשוב לגבי מצב השיעור (שינוי בחדר, ביטול). התלמידים משתמשים בה כאמצעי משלים לשוב לצורך קבלת עדכונים. בנוסף לבעיה שלא הכל במקום אחד גם לאפליקציה הזאת יש טיפה בעיות שאפשר לפתור. המערכת אינה מותאמת אישית – השיעורים מוצגים לפי שכבות ומגמות, מה שמקשה על התלמיד להבין אילו שיעורים רלוונטיים לו ובאיזה סדר.

פנים-ה תפתור את הבעיות האלה, בעיקר את הכניסה הארוכה, כמו כן כל תלמיד יראה את המערכת המותאמת לו, לפי הכיתות שהוא נמצא בהם. ובנוסף את האפשרות למורים לעדכן את פרטי השיעור לתלמידי הכיתה.

תיחום וסקירת טכנולוגיות הפרויקט

הפרויקט שלי משתמש בטכנולוגיות פשוטות ונפוצות, אבל שילבתי אותן בצורה שמביאה פתרון אמיתי לבעיה בבית ספר. הכל כתוב בפייתון – גם הממשק, גם השרת, וגם כל הלוגיקה.

הטכנולוגיות שבהן השתמשתי

- תקשורת TCP – socket - בשביל ליצור חיבור בין הלקוח לשרת. זה מאפשר שליחת פקודות ונתונים בצורה אמינה.
- בסיס נתונים - SQLite - מסד נתונים שמאוחסן מקומית בקובץ אחד. דרכו אני שומר את כל המידע – תלמידים, מורים, שיעורים, נוכחות, טוקנים ועוד.
- הצפנה - RSA + FERNET – שילוב של הצפנה סימטרית (Fernet) עם RSA לצורך שליחה בטוחה של מפתח ההצפנה הראשוני. המידע בין השרת ללקוח עובר מוצפן.
- הבטחת נתונים בבסיס נתונים - Hash (SHA-256) - סיסמאות המשתמשים נשמרות כערכים מוצפנים (לא קריאים), כדי שגם אם מישהו ייכנס לבסיס הנתונים – הוא לא יוכל לראות את הסיסמה.
- UUID + Platform - נוצר טוקן מזהה לכל מכשיר לפי מזהה החומרה (MAC ADDRESS), מערכת ההפעלה וסוג המחשב. ככה אני מוודא שזיהוי הפנים הוא מהמכשיר הנכון.
- Tkinter / CustomTkinter - משמש לממשק הגרפי של המשתמש (UI) – התחברות, זיהוי פנים, תצוגת מערכת שעות, כפתורים ועוד. בחרתי ב CustomTkinter- בגלל הגמישות בעיצוב.

- **InsightFace** - ספרייה לזיהוי פנים. היא קוראת מהמצלמה, מזהה את הפנים, ומחשבת – embedding כלומר מזהה מי המשתמש לפי המספרים של הפנים. (עוד על זה בהמשך)
- **Threading** – בשרת, כל לקוח חדש שמתחבר מקבל thread משלו – ככה אפשר להריץ כמה לקוחות במקביל בלי שיתקעו זה בזה.

פירוט תיאור המערכת:

המערכת משמשת להצגת מערכת שעות מותאמת אישית לתלמידים ולמורים, תוך שילוב אפשרות התחברות עם זיהוי פנים. אל המערכת ניתן להתחבר באמצעות תעודת זהות וסיסמה, או באמצעות זיהוי פנים במידה שהמשתמש הגדיר זאת מראש. כל משתמש מזהה גם לפי מכשיר אישי, באמצעות טוקן שמחושב לפי חומרת המחשב ומערכת ההפעלה.

לאחר התחברות, נפתח מסך המערכת הראשי שבו מופיעה מערכת השעות של המשתמש – תלמידים רואים את השיעורים שלהם לפי הכיתות שבהן הם רשומים, ומורים רואים את מערך השיעורים שהם מלמדים. לכל שיעור מוצגים השעה, הכיתה, המקצוע ושם המורה.

בנוסף לכך בראש המסך הכפתור להעלות תמונת פנים עדכנית, המערכת תצלם, תנתח, ותשמור את הנתונים שקיבלה לאותו משתמש בבסיס הנתונים.

למורים קיימת אפשרות לבצע שינויים זמניים בשיעור, כגון ביטול שיעור או שינוי חדר, דרך הממשק. שינויים אלה נראים בצבע שונה אצל התלמידים, והמערכת מחזירה אותם אוטומטית למצב המקורי ביום אחרי השיעור.

פירוט הבדיקות שמתוכננות לפרויקט

- בדיקת התחברות של משתמש קיים:
מטרת הבדיקה: לבדוק אם משתמש עם תעודת זהות וסיסמה נכונים מצליח להתחבר למערכת.
- בדיקת התחברות עם סיסמה שגויה:
מטרת הבדיקה: לוודא שהמערכת מזהה סיסמה שגויה ומציגה הודעת שגיאה מתאימה.
- בדיקת התחברות עם זיהוי פנים:
מטרת הבדיקה: לבדוק אם משתמש שהפנים שלו שמורות במערכת מצליח להתחבר אוטומטית עם מצלמה.
- בדיקת הצגת מערכת שעות מותאמת אישית:
מטרת הבדיקה: לוודא שלכל תלמיד ומורה מוצגת רק מערכת השעות הרלוונטית לו.
- בדיקת שינוי שיעור על ידי המורה: מטרת הבדיקה: לבדוק האם שינוי של חדר או ביטול שיעור מתעדכן ונראה אצל תלמידי השיעור.
- בדיקת הצפנה ואבטחת מידע:

מטרת הבדיקה: לבדוק שכל הנתונים שנשלחים בין הלקוח לשרת מוצפנים כראוי, ואין דרך לגשת אליהם מבחוץ.

- בדיקת יציבות של בסיס נתונים SQL: מטרת הבדיקה: לוודא שכל הנתונים (תלמידים, מורים, שיעורים, זיהוי פנים) נשמרים כראוי, ולא נעלמים לאחר סגירת המערכת.

ניהול זמנים/תכנון עבודה:

כתיבה ב SQL-עם שאילות בטוחות כדי למנוע SQL Injection

התחלת העבודה על הממשק הגרפי עם CustomTkinter בניית מסך ראשי של מערכת השעות

פיתוח מערכת תקשורת בין לקוח לשרת דרך סוקט (TCP) עם שליחת פקודות בפורמט JSON

הוספת שכבת אבטחה – שימוש ב RSA-לשליחת מפתח, Fernet ואז הצפנה של כל ההודעות

בניית מערכת התחברות – גם עם סיסמה (hash) וגם עם זיהוי פנים דרך מצלמה

הפיכת כל פעולת מסד נתונים לעוברת רק דרך השרת (הלקוח לא ניגש ישירות למסד)

יצירת מסך התחברות עם ת"ז וסיסמה או זיהוי פנים

הוספת אפשרות לצלם תמונת פנים ולשמור אותה כ embedding-במסד

יצירת הרשאות – תלמידים רואים את המערכת שלהם, מורים רואים את השיעורים שהם מלמדים

הוספת כפתור למורים לשינוי זמני של חדר או ביטול שיעור

תכנון חלק של נוכחות אוטומטית לפי זיהוי פנים בתחילת שיעור – ירד בפועל בגלל בעיות טכניות

ניהול סיכונים והדרכים להתמודד איתם:

1. אתגר משמעותי בתחילת העבודה היה בניית בסיס הנתונים מהתחלה – כולל תכנון הטבלאות, הקשרים ביניהן, מגבלות מפתח זר, ושמירה על שלמות הנתונים. מכיוון שלא עסקתי קודם ב־SQL ברמה כזאת, הייתי צריך ללמוד תוך כדי הפרויקט איך מוודאים שכל פעולה שנעשית במערכת – כמו שיוך תלמידים לכיתות או עדכון מערכת שעות – תשתקף בצורה נכונה במסד. כל שינוי קטן במבנה גרם לבאגים שהיו קשים לאיתור, ולפעמים אילץ אותי למחוק ולהקים את המסד מחדש. היה צורך להבין גם איך להתמודד עם שגיאות של שלמות מידע, כפילויות, ואיך לעצב את השאילות בצורה שלא תגרום לקריסות

2.

אתגר נוסף היה להעביר תמונות בין הקליינט לסרבר, חוץ מהתמונות של הזיהוי פנים שנשלחים יש לי תמונות של אייקונים שאני מעביר בשביל הUI. אותם פשוט הקטנתי שלא יעברו כמוט ביטים מסוים וכך יכולתי להמשיך להעביר כרגיל. אך כשהגיע הזמן להעביר תמונה של הפנים בשביל לזהות, לא ניתן להקטין (כלומר להוריד את האיכות) כי אנחנו צריכים תמונה חדה להשוואות לזאת שבבסיס נתונים (או לפחות embedding שלה).

בשביל לעבור את המכשול הזה, הייתי צריך לעשות לולאה שמקבלת את המידע עד כתב של <<END>> (שצירפתי לכל צד כשהוא שולח את המידע), שם אדע שהסתיים ההודעה לעבור בין המשתמש לסרבר.

בשיטה הזאת אני יכול להעביר כמות גדולה מאוד של הודעות בדיוק כמו התמונה באיכות גבוהה, שאותה גם הצפנו (מדובר בכמה עשרות אלפי תווים).

3.

אתגר נוסף שהתמודדתי איתו היה בהצפנת המידע שעובר בין הלקוח לשרת, במיוחד כשהתחלתי להעביר קבצים כבדים כמו תמונות פנים לצורך זיהוי. ניסיתי להשתמש ב-RSA כי הוא נחשב בטוח, אבל מהר מאוד גיליתי שהוא פשוט לא מתאים להצפנה של מידע בנפח גדול — הוא מוגבל לאורך קצר מאוד ומקריס הכל אם מנסים להצפין איתו קובץ גדול. הפתרון שמצאתי היה לשלב בין שני סוגי הצפנה: השתמשתי ב-RSA כדי להצפין מפתח זמני של Fernet, ואז את כל שאר ההודעות — כולל תמונות כבדות — הצפנתי עם Fernet, שהוא מהיר ויעיל להצפנת כמויות גדולות של מידע. ככה הצלחתי לשמור על אבטחה גבוהה בלי לוותר על העברת תמונות באיכות מלאה.

תיאור תחום הידע – פרק מילולי

שם היכולת: כניסה באמצעות זיהוי פנים

מהות היכולת:

מאפשרת לתלמיד להיכנס למערכת באמצעות צילום פנים בלבד, ללא צורך בסיסמה או תעודת זהות. התמונה שנלכדת נשלחת לשרת, משווית ל-embedding השמור בבסיס הנתונים, ואם נמצא דמיון גבוה – המשתמש מזוהה ומחובר.

אוסף יכולות נדרשות:

- קליטת תמונה - שמירת תמונה זמנית של פני המשתמש.
- המרת תמונה - התמונה מומרת ל-embedding (וקטור 512 ממדים)
- שליחה לשרת - העברת embedding דרך socket כשהיא מוצפנת.
- השוואה בבסיס הנתונים – השוואת embedding מול אלה השמורים.
- קבלת תשובה - קבלת תוצאה מהשרת: האם נמצא משתמש מתאים.
- הצגת תוצאה - מעבר למסך הראשי במקרה של זיהוי מוצלח.

אובייקטים נחוצים:

- ממשק משתמש (CTk)
- מצלמה ועיבוד תמונה (cv2, insightface)
- תקשורת סרבר קליינט
- בסיס נתונים (SQLite)

שם היכולת: שליחת פקודה לשרת

מהות היכולת:

מאפשרת ללקוח (Client) לשלוח פקודה מסוימת לשרת (Server) בפורמט מסודר הכולל שם פונקציה וארגומנטים. משמשת כתווך כללי בין פעולות ה-UI לבצוע בצד השרת.

אוסף יכולות נדרשות:

- **בניית מבנה JSON** - הכנסת שם הפונקציה וה-args מביתך מילון.
- **הצפנה** - הצפנת המילון באמצעות Fernet.
- **שליחה לשרת** - שידור ההודעה המוצפנת דרך socket.
- **קבלת תגובה מהשרת** - קבלה ופענוח של תשובה מוצפנת.

אובייקטים נחוצים:

- תקשורת סרבר קליינט
- ספריית הצפנה (FERNET)
- Json (עיבוד נתונים)
- Client trigger (פעולה כלשהיא בממשק או קריאה בקוד)

שם היכולת: עיבוד בקשת לקוח:

מהות היכולת:

קבלת בקשות מוצפנות מהלקוח, פענוח הפקודה, הרצת הפונקציה המבוקשת בצד השרת, ושליחת תשובה מוצפנת חזרה ללקוח. כולל ניהול תקשורת מאובטחת והגנה בפני עומס (DDoS) על ידי הגבלת חיבורים במקביל.

אוסף יכולות נדרשות:

- קבלת חיבור חדש מהלקוח וניהול thread נפרד עבורו.
- שליחת מפתח ציבורי ללקוח, ופיענוח מפתח Fernet מוצפן באמצעות חישובי RSA ידניים.
- קבלת פקודות מוצפנות (JSON) מהלקוח ופענוחן.
- הפעלת פונקציות בצד השרת לפי שם הפונקציה והארגומנטים.
- החזרת התוצאה ללקוח אחרי הצפנה מחדש.
- ניהול עומסים – הגבלת מספר חיבורים בו זמנית למניעת התקפות.
- ניהול חריגות – טיפול בשגיאות במידע שהתקבל או בפונקציות הרצה.
- סיום תקשורת תקין לאחר ניתוק הלקוח.

אובייקטים נחוצים:

- תקשורת socket, threads
- הצפנה fernet, RSA
- מוד נתונים: קריאות ל-Database.py
- ספריות אחרות: json, cryptography, os

שם היכולת: התחברות עם תעודת זהות וסיסמה ועדכון טוקן:

מהות היכולת: אימות פרטי התחברות של המשתמש וביצוע שיוך של הטוקן של המכשיר הנוכחי רק אליו, תוך ניתוק חשבונות אחרים שהשתמשו באותו מכשיר.

אוסף יכולות נדרשות:

- קליטת פרטי התחברות מהממשק (תז, סיסמה וטוקן)
 - הצפנת הסיסמה בצד לקוח עם hashing
 - שליחה מאובטחת לשרת באמצעות Fernet
 - בדיקת תקינות מול בסיס הנתונים (התאמה בין תז וסיסמה שעברה hash)
 - שיוך הטוקן הנוכחי לחשבון שהתחבר
 - סגירת טרנזקציה בהצלחה (commit) או ביטול (rollback) במקרה של שגיאה
 - שליחת תשובה מוצפנת ללקוח
-

אובייקטים נחוצים:

בסיס נתונים, אבטחת מידע, (hashing) מנגנון תקשורת, הצפנה, (Fernet)

מבנה הפרויקט

תיאור ארכיטקטורת המערכת:

לקוח (CustomTkinter+ client)-

הלקוח אחראי על כל מה שהמשתמש רואה ומפעיל – הוא גם יוצר את החיבור לשרת, גם שולח את הבקשות, וגם מציג את התשובות מהשרת במסכים גרפיים. בפועל, החלק הזה מורכב משני קבצים:

- **Client.py** – זה הקובץ שמנהל את התקשורת עם השרת. הוא יוצר חיבור TCP ל-localhost, שולח את הפקודות בפורמט JSON כשהן מוצפנות מראש עם Fernet ומקבל את התשובות המוצפנות בחזרה. הוא גם מבצע את ההצפנה הראשונית של המפתח עם RSA ושומר טוקן מזהה לכל מכשיר.

- **customTkinter.py** – זה החלק של הממשק. הוא מציג את כל המסכים: כניסה, מערכת שעות, שגיאות, טעינה, כפתורים ועוד. כל פעולה של המשתמש דרך המסך (כמו התחברות או מעבר יום) מפעילה פונקציה ששולחת פקודה לשרת דרך Client.py ומחכה לתוצאה. הממשק הזה נבנה עם ספריית CustomTkinter כדי לשמור על עיצוב מודרני ונוח.

שני הקבצים עובדים יחד Client.py: מטפל מאחורי הקלעים, ו- CustomTkinter.py מציג את זה בצורה גרפית ונוחה למשתמש.

שרת (Server.py):

השרת הוא המרכז של כל המערכת – הוא מקבל פקודות מהלקוח, פותח את מסד הנתונים, עושה את הפעולה שביקשו ממנו, ושולח את התוצאה חזרה ללקוח, הכל עובר בצורה מוצפנת.

ברגע שלקוח מתחבר, השרת שולח לו את המפתח הציבורי (RSA), ומצפה לקבל חזרה מפתח Fernet מוצפן. משם כל התקשורת ממשיכה רק עם Fernet מוצפן.

כדי למנוע עומס או תקיפה (DDOS), השרת מגביל את מספר החיבורים הפתוחים בו זמנית ל-15 לקוחות. כל חיבור כזה מקבל thread נפרד, כדי שכל המשתמשים יוכלו לפעול במקביל בלי לחסום אחד את השני.

רוב הפקודות בשרת מטופלות בצורה דינמית – כלומר, כל פקודה שהלקוח שולח מכילה את שם הפונקציה שצריך להפעיל ואת הפרמטרים שלה, והשרת פשוט קורא לפונקציה הזאת (בדרך כלל מתוך Database.py), עם הפרמטרים שנשלחו. עם זאת, יש גם כמה פקודות מיוחדות שמטופלות ידנית כמו face_id_login או שליחה של אייקונים ל-GUI.

מסד נתונים (Database.py):

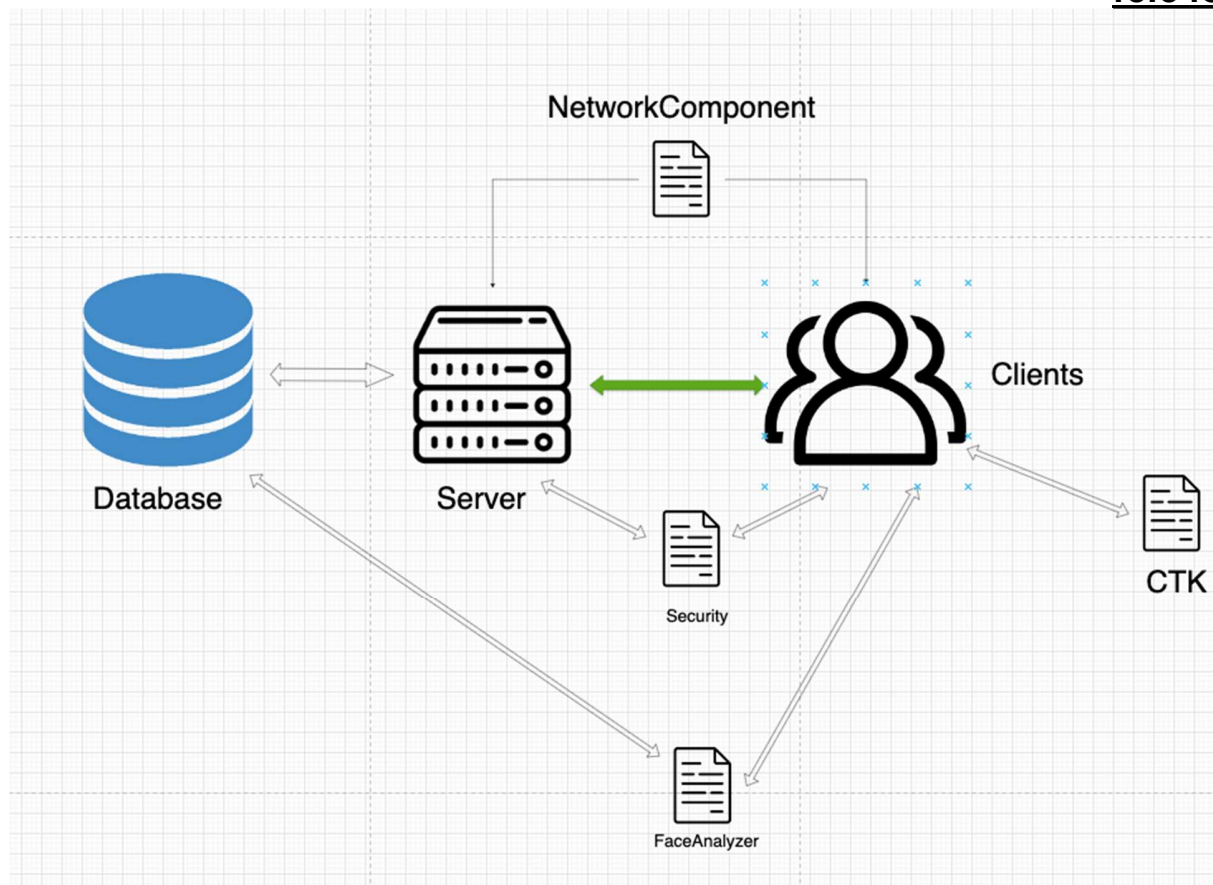
מסד הנתונים בנוי בקובץ SQLite מקומי, והוא מכיל את כל המידע הדרוש להפעלת המערכת – תלמידים, מורים, שיעורים, לוגין, טוקנים, ומידע על שינויים במערכת השעות.

ברוב הבקשות במסד יש שימוש בחיבורים בין טבלאות, לדוגמה, כשמחפשים מערכת שעות של תלמיד או מורה, צריך לחבר בין שיעורים, תלמידים, מורים, ועוד. רוב הטבלאות במסד מחוברות אחת לשנייה, ככה שכמעט כל שאילתה מרכזית שולפת נתונים מכמה מקומות.

הגישה למסד מתבצעת רק דרך הפונקציות שב־ Database.py, שמאפשר שליטה על מה יוצא ונכנס ומונע גישה ישירה או בעיות של אבטחה. כל Thread בשרת פותח חיבור נפרד למסד.

חשוב לדעת שחוץ מה־4 קבצים שדיברתי עליהם כאן, יש עוד שלוש קבצי עזר: הראשון Security, משתמשים גם הקליינט וגם הסרבר, הוא מצפין הודעות בפורמט של json מכיוון שכל מה שאני מעביר הוא dict. המחלקה השנייה היא FaceAnalyzer, בה מרוכז פונקציות שקשורות לזיהוי פנים כמו צילום תמונה ותהליך הזיהוי עצמו שנעשה על ידי ספרייה insightface. והשלישי הוא NetworkComponent שממנו יורשים גם הסרבר וגם הקליינט.

שרטוט:



תיאור הטכנולוגיות הרלוונטיות:

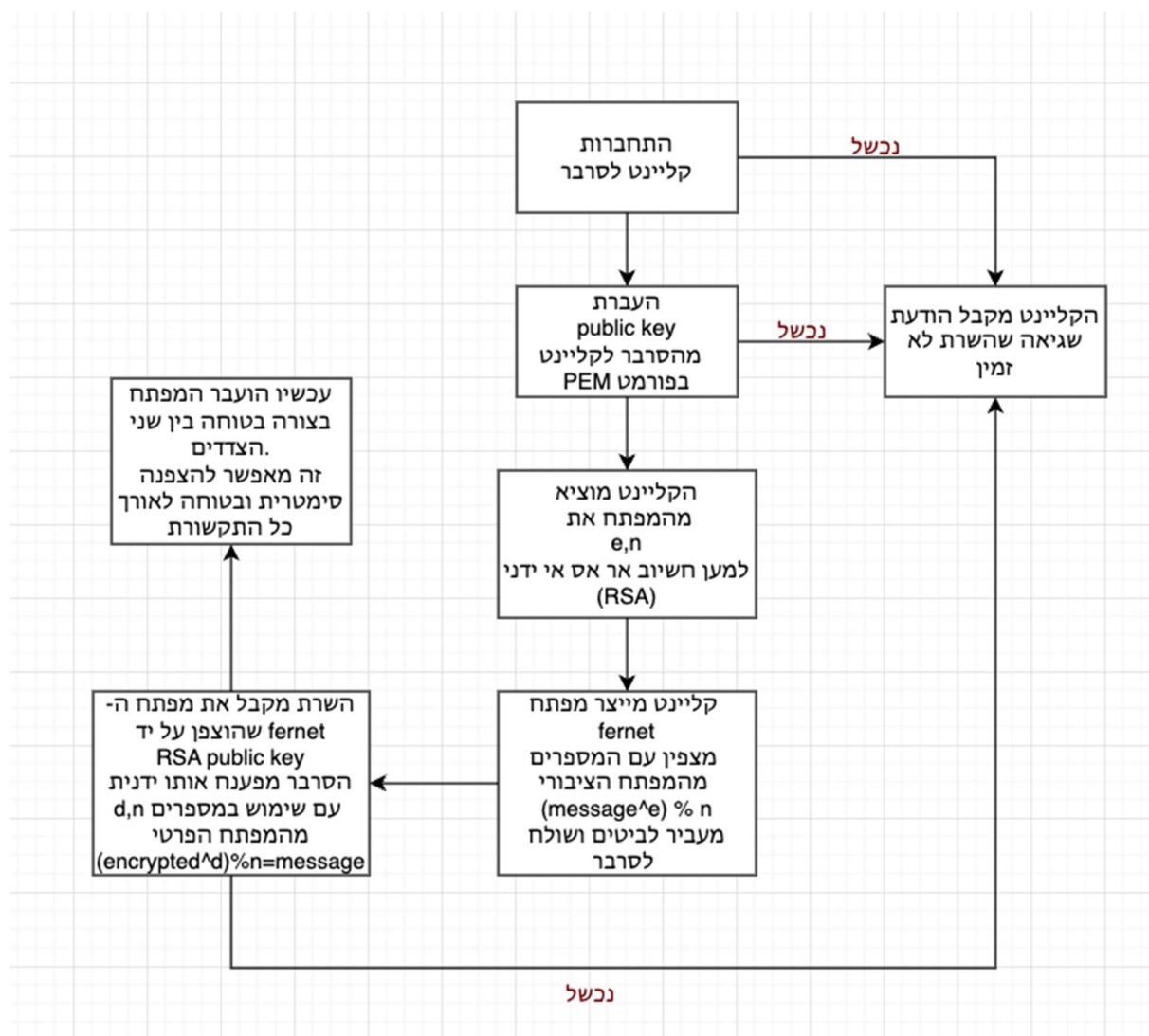
שפת תכנות – פיתון: שפה פשוטה ונוחה לעבודה, שקל להבין ולכתוב בה גם כשעובדים על מערכת שלמה. השתמשתי בה כי היא מתאימה לפרויקטים מהסוג הזה – עם לקוח, שרת, מסד נתונים, הצפנה, GUI, מצלמה ועוד. יש לה הרבה ספריות שעזרו לי לממש כמעט כל חלק בפרויקט בצורה יעילה ומהירה.

מערכת הפעלה – MacOS – מערכת הפעלה של המחשב שלי היא MacOS של Apple. אך בנוסף בעבודה בבית ספר השתמשתי במערכות Windows 11.

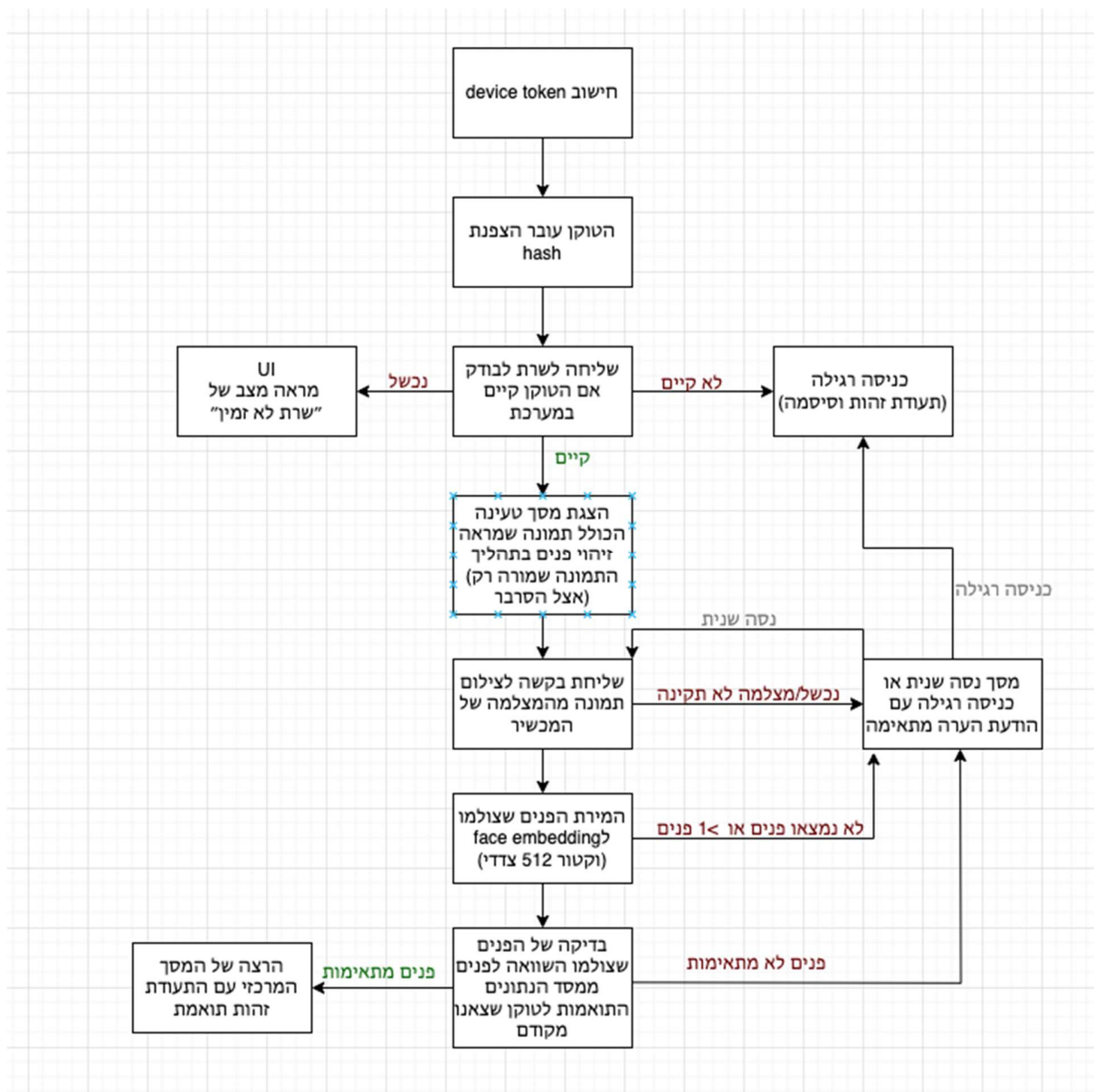
תקשורת – התקשורת בין הלקוח לשרת מתבצעת באמצעות פרוטוקול TCP. בחרתי ב-TCP כי הוא חיבור יציב ומובטח – כל הודעה שנשלחת מגיעה בשלמותה ובסדר הנכון, מה שחשוב במיוחד כשמעבירים מידע מוצפן או נתונים רגישים כמו פרטי התחברות ותמונות פנים.

בסיס נתונים – database – בשביל הפרויקט השתמשתי ב-SQL על מנת לאחסן ולשלוח נתונים.

תרשים זרימה Hybrid Encryption:



תהליך כניסה באמצעות זיהוי פנים:



תהליך כניסה רגילה:



[illegible]

אלגוריתמים:

אלגוריתם זיהוי פנים:

לצורך כניסה באמצעות זיהוי פנים, וגם בהמשך לצורך סימון נוכחות באמצעות זיהוי פנים צריך א. למצוא פנים בתמונה, ב. "לזהות" אותם. בפועל כאשר אנחנו רוצים להשוואות בין 2 פנים (זה שמצאנו בתמונה וזה שידועה לנו מראש), לא ניתן להשוואות ביניהם ישירות כמו לדוגמה $face1 == face2$. לכן צריכים קודם לכן לפרק את הפנים למה שידוע כ- embedding, וקטור 512 צדדי המייצג את הפנים. לכן צריך ספרייה שתתן לנו את האפשרות לעשות זאת, בצורה הכי מדויקת ומהירה שאפשר. 2 האלגוריתמים שניסיתי בפרויקט היו 1. face_recognition. 2. insightface. מקורות –

<https://insightface.ai>

<https://face-recognition.readthedocs.io>

הספרייה שאני בחרתי בה היא insightface. לאחר ניסוי רב עם הספרייה face_recognition, הבנתי שהיא נותנת יותר שליטה לגבי תהליך המציאה וזיהוי פנים (כגון: scale factor, face finding, threshold...). וגם מקלה על השוואה בין 2 הווקטורים- אך היא הרבה פחות מדויקת מספריית insightface. בפרויקט שלי לא רציתי שיקרה מצב שתהליך הזיהוי ייכשל בגלל טעות של המערכת, אלא רק אם באמת הפנים לא תואמות למשתמש. לכן העדפתי דיוק גבוה יותר על נוחות, ובחרתי בinsightface.

אלגוריתם הצפנה ושליחת הודעות:

בפרויקט שלי אני צריך להעביר הודעות בין השרת ללקוח, וההודעות האלו כוללות מידע רגיש כמו סיסמאות, תעודות זהות ותמונות של הפנים. לכן חשוב שהכל יהיה מוצפן, אבל גם יועבר במהירות, כי מדובר בכמויות מידע לא קטנות, בעיקר בתהליך של זיהוי פנים. כדי להגיע לשילוב של מהירות ואבטחה בחרתי להשתמש בהצפנה היברידית, כלומר שילוב של הצפנה אסימטרית מסוג RSA להצפנת מפתח זמני, והצפנה סימטרית מהירה מסוג Fernet לכל שאר ההודעות.

RSA מבוסס על עיקרון מתמטי פשוט אבל חזק: מתחילים מבחירה של שני מספרים ראשוניים גדולים, מכפילים אותם ומקבלים מספר בשם n. את n כולם יכולים לראות, אבל רק מי שמחזיק את המספרים המקוריים יכול לחשב את המפתח הפרטי. מפתח ההצפנה הציבורי כולל את n ואת מספר נוסף שנקרא e, שוברת שומרים את המפתח הפרטי שכולל את אותו n ואת מספר d שמחושב לפי כלל מודולרי. כשרוצים להצפין הודעה, מעלים אותה בחזקת e מודולו n. כשהשרת מקבל את ההודעה, הוא מפענח על ידי העלאה בחזקת d מודולו n.

הבעיה של RSA היא שהוא איטי, ומוגבל בגודל ההודעות שאפשר להצפין איתו. לכן השתמשתי בו רק פעם אחת – בשביל לשלוח לשרת מפתח Fernet. Fernet הוא מפתח באורך 256 ביט שמתאים להצפנה סימטרית מהירה מאוד, בעזרת אלגוריתם AES. הוא גם כולל וקטור אתחול רנדומלי וחתימה דיגיטלית שמבטיחה שלא שינו את התוכן בדרך. כל ההודעות שעוברות בין הלקוח לשרת מוצפנות עם אותו מפתח, Fernet, והן כוללות גם JSON, גם נתונים מבסיס הנתונים, וגם קבצים של תמונות. בגלל שהמפתח הזה נשלח בצורה מוצפנת בהתחלה עם RSA, אפשר להיות בטוחים שהוא לא נגנב, ולכן כל ההמשך יכול להשתמש רק בו בצורה סימטרית, מה שמאפשר מהירות גבוהה בלי לוותר על אבטחה.

בנוסף לכך, הסיסמאות לא עוברות בכלל בין הצדדים בצורה גלויה – גם לא בהתחלה. עוד לפני ההצפנה עם Fernet הסיסמה עוברת דרך פונקציית hash מסוג SHA256 זו פונקציה מתמטית שמקבלת מחרוזת מכל סוג, ומחזירה מחרוזת קבועה באורך 256 ביט, באופן חד-כיווני. כלומר אי אפשר לשחזר את הסיסמה המקורית מתוך התוצאה, וגם שינוי של תו אחד לגמרי משנה את כל הפלט. בצורה כזאת, גם אם משהו פורץ לבסיס הנתונים, הוא לא יוכל לדעת את הסיסמאות של המשתמשים.

השילוב של RSA, Fernet ו-SHA256 נותן לי גם מהירות, גם אבטחה גבוהה, וגם פרטיות מקסימלית – וכל זה בלי לפגוע בחוויית המשתמש או בזמן טעינה.

קישורים:

[https://en.wikipedia.org/wiki/RSA_\(cryptosystem\)](https://en.wikipedia.org/wiki/RSA_(cryptosystem))
https://en.wikipedia.org/wiki/Elliptic-curve_cryptography
<https://cryptography.io/en/latest/fernet/>

תיאור סביבת הפיתוח:

Python 3.10
 Miniconda 3.10
 Pycharm community Edition

בנוסף לכך צריך לוודא שהקבצים של private.pem ו public.pem קיימים בdirectory של השרת, מצלמה תקינה אצל המשתמש, Terminal/CMD ספריית onxruntime, gcc/wheel ו builder tools C++ להרצת מודלים של insightface.

פרוטוקול תקשורת:

פרוטוקול התקשורת בה השתמשתי בפרויקט שלי היא TCP בחיבור מסוג socket. TCP נמצא בשכבה הרביעית ומאבטח העברת מידע אמינה של המידע בין שני הצדדים (קליינט וסרבר). המידע מחולק לחבילות נתונים עם מזהים כמו מספר סידורי ואישורים, כך שהמידע מובטח להגיע בצורה תקינה ובסדר הנכון גם אם השתבש בדרך. דבר כזה נקרא חיבור Connection oriented.

הסיבה שהשתמשתי בפרוטוקול הזה ולא בUDP, המאפשר העברת מידע מהיר יותר במחיר של צורה פחות בטוחה, כלומר שהמידע לא בטוח להגיע ולא בסדר הנכון- הוא מכיוון שאין אצלי בפרויקט מידע שאמור להגיע הכי מהר שאפשר, כמו סרטון או מספר תמונות רבות.

מאגר הודעות:

1. שם ההודעה: public key exchange
נשלח מ/אל: מהשרת ללקוח
השרת שולח את המפתח אר אס אי הציבורי מקודד אבל ללא הצפנה כי זה בטוח לעשות זאת.
2. שם ההודעה: fernet key exchange
נשלח מ/אל: לקוח לשרת
הלקוח משתמש במפתח הציבורי שניתן לו ומצפין את המפתח דו סימטרי שיצר.

3. שם ההודעה: find_device_token
נשלח מ/אל: לקוח לשרת
מעביר את הארגומנטים: hashed_device_token. מטרת ההודעה למצוא בבסיס הנתונים אם קיים הטוקן הזה בטבלת logins.
4. שם ההודעה: find_device_token -**response**-
נשלח מ/אל: השרת ללקוח
הודעה חוזרת לגבי הבקשה בפורמט python dictionary: {"status": ..., "output": ...}
5. שם ההודעה: get_face_id_logo
נשלח מ/אל: הלקוח לשרת
בקשה לשרת לקבל ביטים של תמונה
6. שם ההודעה: get_face_id_logo -**response**-
נשלח מ/אל: השרת ללקוח
השרת פותח את התמונה בביטים ומחזיר בפורמט python dictionary: {"status": ..., "output": ...}

7. שם ההודעה: face_id_login
נשלח מ/אל: מהלקוח לשרת
הלקוח מבקש מהשרת לבצע את תהליך הזיהוי פנים עם הארגומנטים של ביטים התמונה שצולמה, ותעודת זהות בשביל להשוואת עם מה שנמצא במסד נתונים
8. שם ההודעה: face_id_login -**response**-
נשלח מ/אל: השרת ללקוח
השרת מבצע את הפקודה שהתבקש ממנו בקובץ של מסד הנתונים ומחזיר תשובה בפורמט של python dictionary: {"status": ..., "output": ...}

9. שם ההודעה: get_back_icon

נשלח מ/אל: הלקוח לשרת
בקשה לשרת לקבל ביטים של תמונה

10. שם ההודעה: **-response-get_back_icon**
נשלח מ/אל: השרת ללקוח
השרת פותח את התמונה בביטים ומחזיר בפורמט python dictionary: {"status": ..., "output": ...}

11. שם ההודעה: **register_face_embedding**
נשלח מ/אל: הלקוח לשרת
נשלח עם הארגומנטים של ת"ז המשתמש, סוג (תלמיד או מורה), וביטים התמונה שצולמה.
מבקש מהשרת לבצע את הפקודה במסד נתונים.

12. שם ההודעה: **-response-register_face_embedding**
נשלח מ/אל: השרת ללקוח
השרת מבצע את הפקודה שהתבקש ממנו בקובץ של מסד הנתונים ומחזיר תשובה בפורמט של python dictionary: {"status": ..., "output": ...}

13. שם ההודעה: **temporary_change**
נשלח מ/אל: הלקוח לשרת
נשלח עם כל הארגומנטים שנדרשים לעדכן נתונים של שיעור, הכוללים מספר חדר, סטטוס השיעור, וכל המאפיינים של השיעור שיהיה אפשר למצוא אותו במאגר.

14. שם ההודעה: **-response-temporary_change**
נשלח מ/אל: השרת ללקוח
השרת מבצע את הפקודה שהתבקש ממנו בקובץ של מסד הנתונים ומחזיר תשובה בפורמט של python dictionary: {"status": ..., "output": ...}

15. שם ההודעה: **get_student_schedule/get_teacher_schedule** (אותו עיקרון בדיוק)
נשלח מ/אל: הלקוח לשרת
מבקש מהשרת לבצע את הפקודה במסד נתונים ולהחזיר את שעות המערכת של אותו תלמיד או מורה

16. שם ההודעה: **-response-get_student_schedule/get_teacher_schedule**
נשלח מ/אל: השרת ללקוח
השרת מבצע את הפקודה שהתבקש ממנו בקובץ של מסד הנתונים ומחזיר תשובה בפורמט של python dictionary: {"status": ..., "output": ...}

תיאור מסכי המערכת:

מסך 1-
כאשר נפתח האפליקציה בפעם הראשונה על מכשיר חדש (מכשיר שהdevice token שלו לא קיים במערכת)

התחברות למשוב

תעודת זהות

סיסמה

show

התחבר

ניתן להחביא/לראות את
הסיסמה שמקלידים
(מ**** ל כתב נורמלי)

אחרי כניסה מהמסך
הזה, המערכת תכין
ותשמור את הטוקן של
המכשיר הזה ותקשר
אותו לתעודת זהות
שאינה נכנס המשתמש
בטבלאות הרלוונטיות.

מסך 2-



מסך המתנה לתהליך זיהוי
הפנים כאשר **כן** קיים טוקן
תואם במערכת

מסך 3-

מסך שגיאה/כשל תהליך
זיהוי פנים, קורה כאשר
הפנים לא תואמות, ישנם
יותר מפנים אחת, כשל
כלשהיא של המצלמה ועוד
כמה מקרים.

ניתן במסך הזה ללחוץ על
אחד מהכפתורים,
הראשון -נסה שוב-
תחזיר אותנו למסך השני
בו התהליך יתחיל מחדש
ויתן למשתמש עוד
הזדמנות.

השני -כניסה רגילה-
יחזיר אותנו למסך הראשון
בו יתאפשר להיכנס כרגיל
עם תעודת זהות וסיסמה.

התחברות למשוב



זיהוי פנים בתהליך...

זיהוי הפנים נכשל, נסה שוב.

נסה שוב

כניסה רגילה

מסך 4-

העלה תמונת פנים		
ראשון	שני	שלישי
רביעי	חמישי	שישי
	7:45 8:30	0
	8:30 9:15	1
108 סטלה מורה	מתמטיקה 5 יח"ל	9:15 10:00
107 סטלה מורה	מתמטיקה 5 יח"ל	10:20 11:05
107 סטלה מורה	מתמטיקה 5 יח"ל	11:05 11:50
111 בוריס	מדעי מחשב מבני נתונים	12:05 12:50
111 בוריס	מדעי מחשב מבני נתונים	12:50 13:55
	15:55	7

המסך המרכזי במערכת, מראה את המערכת של כל יום, ניתן להחליף בין הימים למעלה.

ניתן לראות שכאשר מורה מבצע שינוי בשיעור (מסך של זה בעמוד הבא), התלמיד יראה את השינויים האלה, מספר חדר שונה מהרגיל יהפוך לצהוב, שיעור שהתבטל נהפך לאפור.

הכפתור למעלה מוביל למסך (מסך 6) של העלאת תמונה עדכנית של הפנים לעדכן במסד נתונים (embedding הפנים, לא התמונה עצמה)

מסך 5-

מסך לשינוי פרטי השיעור,
בנוי רק אצל המערכת של
המורים, בלחיצה על
השיעור עצמו נפתח המסך
הזה.
ניתן לבטל ולשנות את
מספר החדר, לאפס
שינויים (הכפתור יופיע רק
אם יש שינויים לאפס)

התמונה למעלה היא
כפתור לחזור למסך
המרכזי.



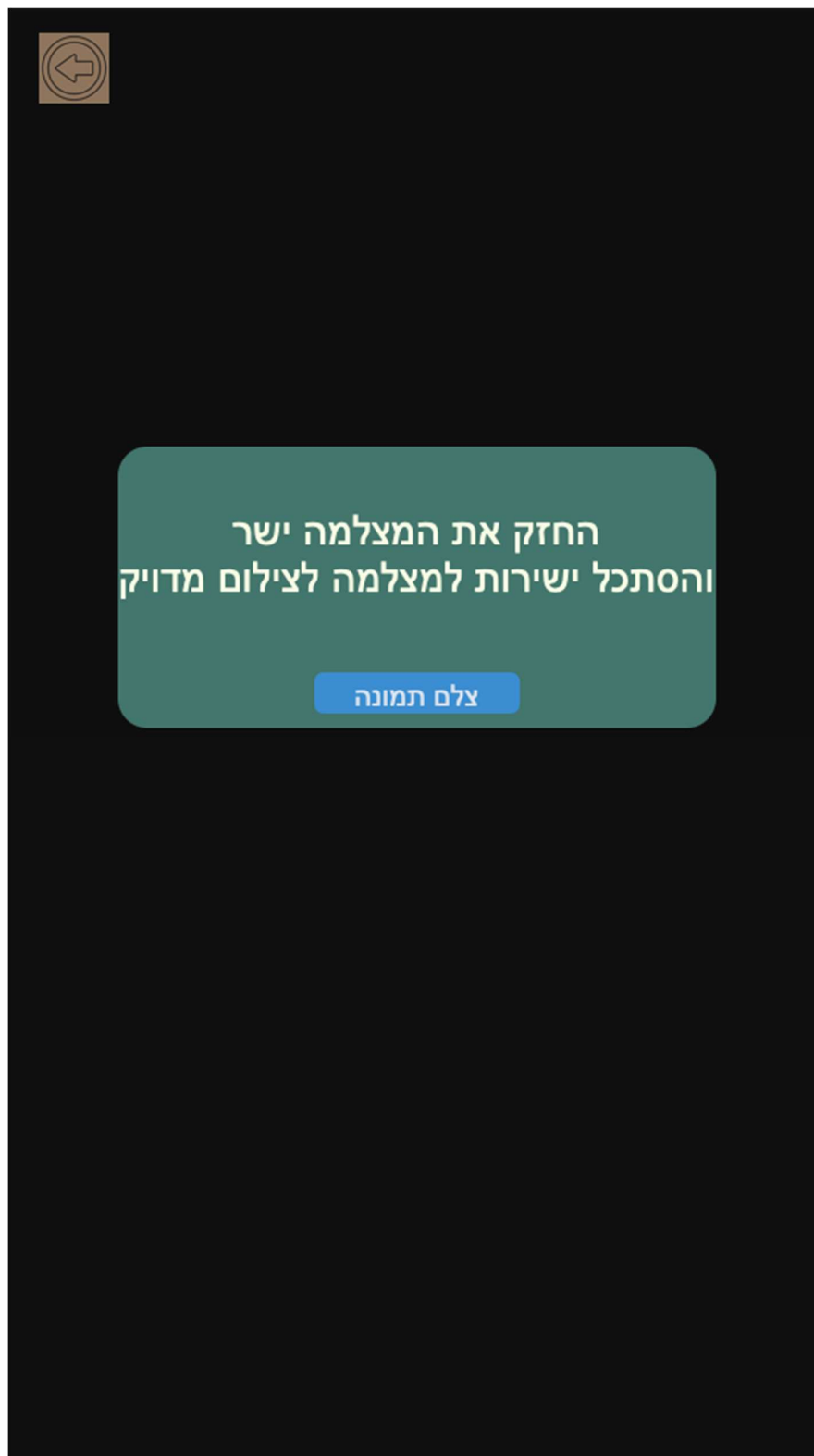
הכנס מספר כיתה חדש (רשות)

לדוגמה: 203

☐ בטל שיעור זה

בצע

איפוס שינויים



מסך 6-
מסך שמופיע בלחיצה על
הכפתור בראש המסך
המרכזי (מסך 4).

כאן ניתן לעדכן תמונה
של הפנים למסד נתונים
(העדכון הוא של
embedding הפנים ולא
תמונה של הפנים עצמם)

ניתן לחזור אחורה למסך
המרכזי בלחיצה על
התמונה למעלה.

לאחר לחיצה על צילום
תמונה תופיע הדועה
בהתאם-

"שגיאה: לא נמצאה
תמונה תקינה, אין פנים
או שיש יותר מאחד"

או
"תמונה נשמרה
בהצלחה!"

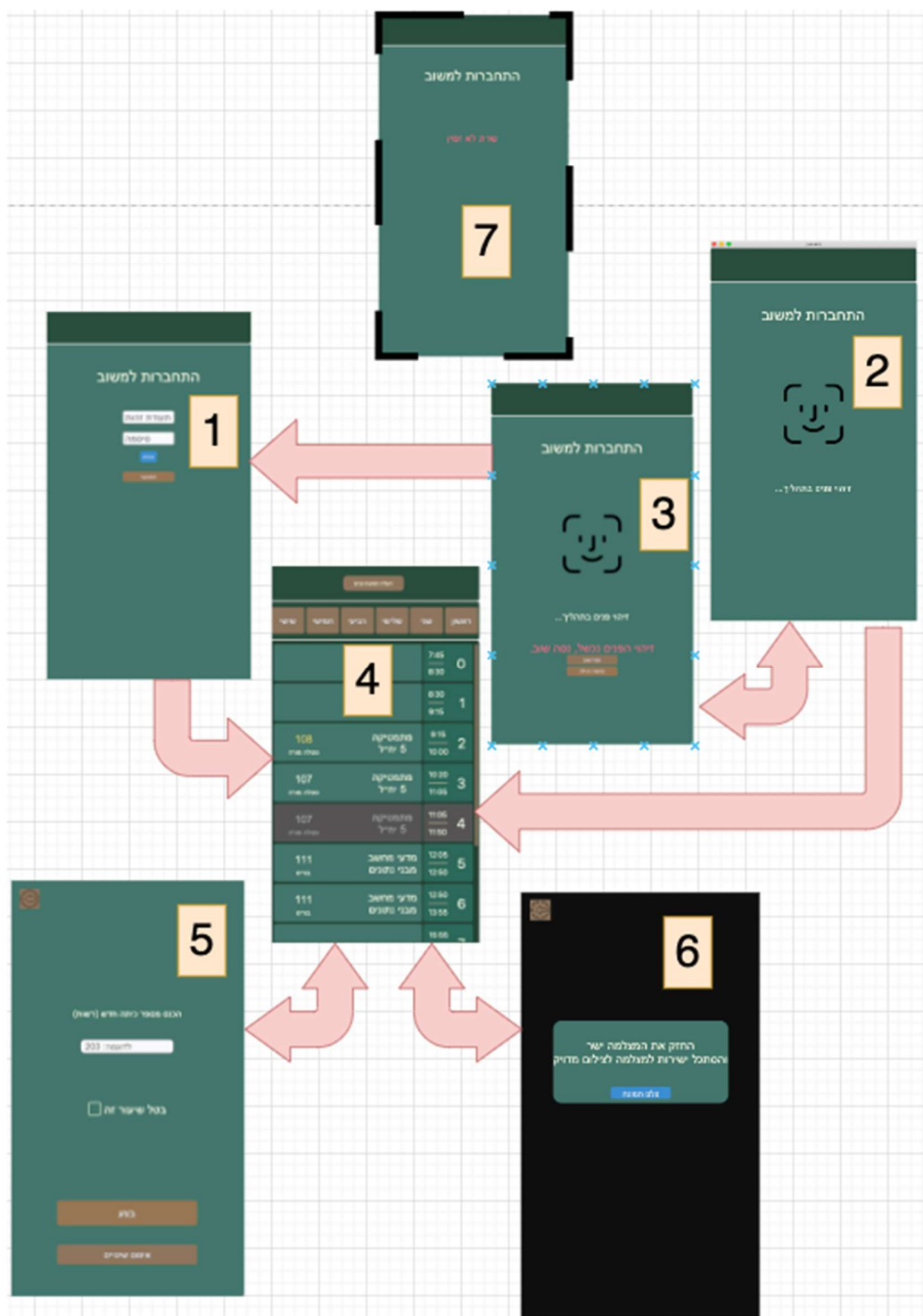
מסך 7-

במצב של שרת לא זמין מכל
סיבה שהיא, יופיע למשתמש
המסך הזה.

התחברות למשוב

שרת לא זמין

תרשום זרימה של מסכי המערכת:
(המספרים הם אלה שאיתם הצגתי את המסכים לא בהכרח מספר סידורי)



תיאור מבני הנתונים:

קבצי המפתחות `public.pem`, `private.pem`:

מכילים את מפתחות ההצפה RSA שבו נעשה שימוש כאשר מתחבר משתמש לסרבר, איתו אנחנו מעבירים את המפתח fernet שנוצר כל פעם מחדש אצל של משתמש. המפתחות אצל השרת ולא נגישים ללקוח, במקום זה בתחילת ההתחברות, השרת מעביר את המפתח הציבורי למשתמש, כדי שיוכל להצפין אבל לא לפענח עדיין הודעות, כך נשמרת אבטחת ההעברה.

קבצי תמונות ממשק-

:Face_id logo.png

סוג קובץ- .png
מאוחסן בשרת ונשלח ללקוח בעת בקשה `get_face_id_logo`, שימוש התמונה לייצג למשתמש תהליך של זיהוי פנים.

:Back icon.png

סוג קובץ- .png
מאוחסן בשרת ונשלח ללקוח בעת בקשה `get_back_icon`, שימוש התמונה לייצג כפתור "חזור" במסכים הנפתחים מהמסך הראשי של המערכת.

פירוט מסד נתונים database:

שם הקובץ- `database.py`.

מסד הנתונים משמש לאחסון קבוע של מידע על משתמשים, שיעורים, מערכות שעות וזיהוי פנים. המידע נשמר בצורה טבלאית, כאשר לכל טבלה שדות קבועים, טיפוסים מוגדרים, ומפתח ראשי. המערכת משתמשת במסד זה לביצוע פעולות כמו כניסת משתמשים, שליפת מערכת שעות, רישום זיהוי פנים ועדכון זמני של שיעורים.

אצלי במערכת מסד נתונים יש הרבה קשרים בין הטבלאות, בשביל לא לאחסן יותר מדי בכל טבלה יש מאפיינים UNIQUE ו-FOREIGN KEY לחלק מהנתונים כדי לקשר אותם עם כאלה מטבלאות אחרות. לדוגמה כדי לא לשמור את כל הנתונים של שיעור בנוסף לכל הנתונים של תלמיד, נעשה שימוש בטבלה `StudentClasses`, הטבלה לוקחת תעודת זהות של תלמיד, ומספר סידורי של כיתה (שונה לכל כיתה), ושומרת אותם במסד נתונים נפרד מזה של `Students` ומזה של `Classes`.

פירוט טבלאות:

Students/Teachers (טבלאות שונות אבל אותו עיקרון)

שם השדה	טיפוס	תיאור	דוגמה
Id	TEXT	תעודת זהות של התלמיד או מורה	"209815158"
Name	TEXT	שם התלמיד או מורה	"דניאל בנטוב"
Embedding	BLOB	וקטור 512 צדדי המייצג את הפנים של התלמיד או מורה	"...b'x9a\x12"

Classes

שם השדה	טיפוס	תיאור	דוגמה
Class_id	INTEGER	מזהה כיתה "יחודי"	4
Grade	TEXT	שכבה	"יב"
number	TEXT	מספר כיתה	"3"
Subject	TEXT	מקצוע	"אנגלית 5 יח"ל"
Teacher_id	TEXT	תעודת זהות של המורה המלמד (מפתח זר לטבלת teachers)	"123456789"

StudentClasses

שם השדה	טיפוס	תיאור	דוגמה
Student_id	TEXT	תעודת זהות של תלמיד (מפתח זר ל-Students)	"209815158"
Class_id	INTEGER	מזהה כיתה (מפתח זר ל-Classes)	4

Classes

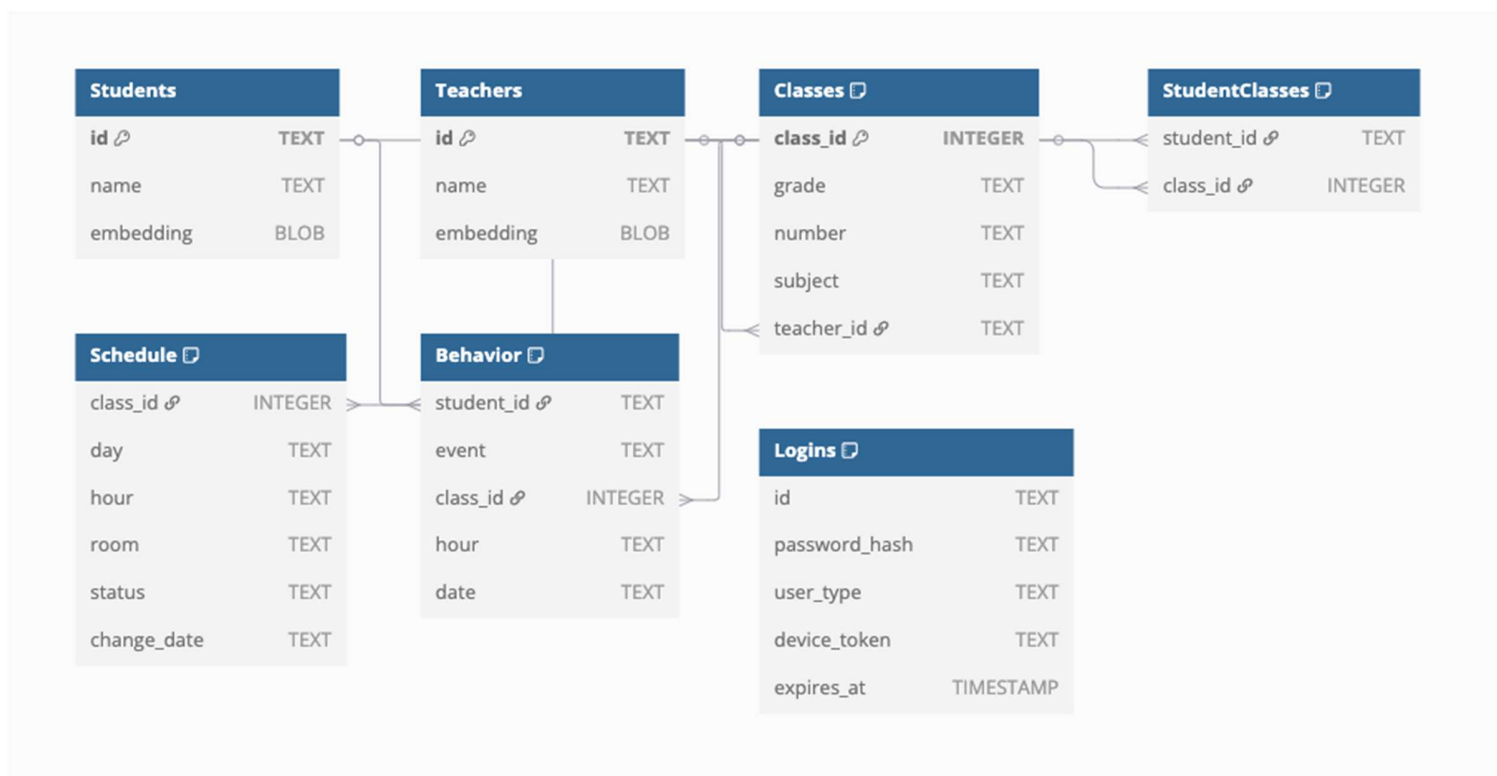
שם השדה	טיפוס	תיאור	דוגמה
Class_id	INTEGER	מזהה כיתה (מפתח זר ל-Classes)	4
day	TEXT	יום בשבוע	"monday"
hour	TEXT	שער שיעור	"3"
Room	TEXT	מספר חדר	"101"
Status	TEXT	סטטוס שיעור (Active/Cancelled)	"Active"
Change_date	TEXT	תאריך שינוי זמני אם יש	"2025-05-25"

Logins

שם השדה	טיפוס	תיאור	דוגמה
Id	TEXT	יום בשבוע	"209815185"
Password_hash	TEXT	שער שיעור	"03ac67..."
User_type	TEXT	מספר חדר	"student"
Device_token	TEXT	מזהה ייחודי של המכשיר הנוכחי	"8c2ec..."

בנוסף יש טבלה behavior בה אעשה שימוש כאשר אבנה את הלוגיקה לנוכחות.

תרשים של הטבלאות:



חולשות ואיומים:

מערכת כזאת יכולה להיות חשופה לכל מיני סוגים של תקיפות וניסיונות לנצל חולשות. לכן, היינו חייבים לחשוב מראש על בעיות שיכולות לקרות ולמנוע אותם כדי שהמערכת תהיה בטוחה.

בשכבת האפליקציה:

1. SQL Injection:

איום: משתמש עלול להעלות קלט עם קוד SQL דרך שדות כמו הכניסה עם ת"ז וסיסמה.
מניעה: שימוש ב-? ופרמטרים, כך שהקוד לא יוכל להתבצע לא משנה מה נכנס כקלט.

2. אימות והזדהות:

איום: התחברות עם פרטי משתמש גנבים או התחזות
מניעה: שימוש בסימאות מאוחסנות עם הצפנת hash ולא בטקסט פתוח.

3. Man in the middle:

איום: יירוט תקשורת בין לקוח לשרת
מניעה: הצפנת הודעות העוברות בין המשתמש לסרבר, בתחלה עם RSA ולאחר מכן עם fernet.

4. DDOS:

איום: עומס פתאומי של התחברויות
מניעה: השרת מקבל מקסימום של 15 משתמשים בכל פעם

בשכבת התעבורה:

פרוטוקול TCP בנוי עם Three way handshake - בהתחלה הלקוח שולח בקשה להתחברות SYN, השרת עונה בחזרה (SYN-ACK) ואז הלקוח מאשר שהוא קיבל (ACK) ככה שניהם יודעים שהצד השני באמת מחובר ושהחיבור תקין.

הצפנה לא מגיעה עם TCP ולכן אני זה שהצפין את ההודעות עם RSA ו-fernet.

מימוש הפרויקט

שם המחלקה Server:

תפקיד-

לנהל תקשורת מול לקוחות, לגשת למסד נתונים לשלוף או לעדכן נתונים, ולהעביר קבצים ששמורים רק מצד השרת כמו תמונות ומפתחות הצפנה.

תכונות-

ip – כתובת הקו של השרת שמאזין לחיבורים.
port – הפורט שבו השרת מקבל את החיבורים
server – אובייקט socket שדרכו השרת מאזין, מקבל ומטפל בחיבורים שנכנסים
S – instance של המחלקה security, אחראי על הצפנה ופענוח בשליחת הודעות.
db – instance של database, אחראי על לקרוא למסד נתונים כשצריך
clients – סופר את כמות הקליינטס שהתחברו כדי למנוע התקפות ddos.

פעולות-

__init__ – פעולה הנקראת כאשר האובייקט נוצר, מגדירה את החיבור למסד נתונים, טוענת מפתחות הצפנה ופותחת את הסוקט

run – מפעילה את השרת בפועל, מאזינה לחיבורים חדשים ופותחת thread לכל לקוח חדש.

handle_client – אחראי על העברת מפתח RSA וקבלת מפתח fernet בחילת תהליך של לקוח ושרת, לאחר מכן מאזינה עם פקודת recv עד קטע הודעה <<END>> ואז ששולחת לטיפול הודעה בprocess message.

process_message – מקבלת את הפקודה שהלקוח שלח, מפרשת אותה ומבצעת בצורה דינמית (חוץ מכמה מקרים בודדים כמו השגה של תמונה מהשרת) את הפונקציה שהלקוח ביקש (לרוב במסד נתונים).
לאחר ביצוע שולחת חזרה הודעה מוצפנת ללקוח עם מה שהוחזר לה ממסד הנתונים (לרוב)

close – סוגרת את החיבור סוקט כשהשרת נסגר

שם המחלקה Client:

תפקיד-

להתחבר לשרת, לבצע את חילופי המפתחות, לשלוח פקודות מוצפנות, לקבל תגובות מוצפנות, ולהריץ את ממשק המשתמש בהתאם לתשובת השרת.

תכונות-

ip- כתובת הִקו של השרת שאליו מתחברים
port- הפורט שאליו הלקוח מתחבר
client- אובייקט socket שמבצע את ההחברות שליחה והאזנה לשרת.
s- instance של המחלקה security, אחראי על להצפין ולפענח הודעות
fernet- מפתח דו סימטרי שנוצר בצד הלקוח ונשלח עם המפתח ציבורי של RSA
hashed_device_token- מזהה ייחודי למכשיר הנוכחי שנוצר ממידע על המערכת
ui- ממשק הגרפי שמופעל לאחר התחברות

פעולות-

- `__init__` – מתחברת לשרת, מקבלת את מפתח ה־RSA שלו, מייצרת מפתח Fernet מצפינה אותו עם RSA ושולחת לשרת.
בונה את ה־UI ומחליטה באיזה מצב להפעיל אותו (offline, face_id, normal) לפי תגובת השרת.
- `send_command(function, args=None)` – מצפין בקשת JSON לפי מפתח ה־Fernet, שולח לשרת, ממתין לתשובה (כולל טיפול ב־`timeout`), מפענח את התשובה ומחזיר אותה כ־dict.
- `run` – מפעיל את הממשק הגרפי של המשתמש.
- `close` – סוגר את החיבור לשרת בצורה מסודרת.

שם המחלקה CustomTkinter:

תפקיד-

לספק ממשק משתמש גרפי בעזרת ספריית CustomTkinter, לבצע התחברות עם סיסמה או זיהוי פנים, ולהציג את מערכת השעות של התלמיד/מורה עם אפשרות לשנות שיעורים (למורה) או לצלם תמונת פנים לעדכון.

תכונות-

client- מופע של המחלקה Client דרכו מתבצעת התקשורת מול השרת
SCREEN_WIDTH וSCREEN_HEIGHT - משתנים קבועים שמגדירים את גודל חלון הממשק
root- חלון ראשי של Tkinter שבו מוצגים כל רכיבי הממשק
כל הצבעים - הגדרות צבעים לממשק
header_frame - מסגרת קבועה בראש המסך עם כפתורים כמו העלאת תמונה
day_to - מציין את היום שמוצג כרגע בלוח השעות

פעולות-

init- פעולה שמוגדרת כשהאובייקט נוצר בונה את חלון הממשק מגדירה צבעים ומסכים ראשוניים
capture_face_image - מצלמת את פני המשתמש ושומרת את הקובץ כקובץ זמני לבדיקת זיהוי
running- מפעילה את ממשק הכניסה ומריצה את חלון ה-Tkinter הראשי
login_ui- מציגה את מסך ההתחברות לפי סוג רגיל או זיהוי פנים כולל טיפול בשגיאות או מעבר למסך הבא בהתאם לתשובת השרת
login_attempt- מתבצעת בלחיצה על כפתור התחברות רגילה בודקת תעודת זהות וסיסמה מול השרת ושולחת גם מזהה ייחודי של המכשיר
schedule- מציגה את מערכת השעות של המשתמש לפי התפקיד שהתקבל מהשרת מורים יכולים לבצע שינויים זמניים בכל שיעור תלמידים יכולים לראות את השיעורים בלבד

שם המחלקה FaceAnalyzer:

תפקיד-

לטעון מודל זיהוי פנים ולשלוף embedding מספרי ייחודי מתוך תמונת פנים כדי לאפשר זיהוי אדם, המחלקה מטפלת גם בתמונות שמגיעות כנתיב וגם כתמונה בביטים

תכונות-

app- מודול של insightface שמכיל את המודל שמזהה פנים ומפיק embedding
buffalo_1 המודל שמופעל הוא ומוכן לעבודה על מעבד רגיל ולא GPU

פעולות-

init-פעולה שמופעלת עם יצירת האובייקט טוענת את מודל buffalo_1 מתוך insightface ומכינה אותו לפעולה
check_webcam_access- בודקת אם מצלמת המחשב זמינה על ידי ניסיון פתיחה וסגירה של VideoCapture
get_embedding- מקבלת קלט שהוא או נתיב לתמונה או ביטים של תמונה אם התמונה תקינה ומכילה פנים מחזירה embedding כמערך מספרים אם לא מחזירה הודעת שגיאה בהתאם למקרה

שם המחלקה Security:

תפקיד-

לבצע הצפנה ופענוח של נתונים בין השרת ללקוח באמצעות ספריית Fernet ולהחזיר או לקבל מידע בצורה מוצפנת בנוסף מבצעת hash של סיסמאות לשם השוואה בטוחה מול מסד הנתונים

תכונות-

לא קיימות תכונות (attributes) באובייקט המחלקה מאחר שכל הפונקציות הן סטטיות וכל השימוש הוא פונקציונלי בלבד

פעולות-

encrypt_json - מקבלת אובייקט מסוג dict ומפתח Fernet מצפינה את המידע ומחזירה מחרוזת מוצפנת
encrypt - מקבלת מחרוזת רגילה ומפתח Fernet מצפינה את המחרוזת
decrypt_json - מקבלת מחרוזת מוצפנת בפורמט json מפענחת אותה ומחזירה dict
decrypt - מקבלת מחרוזת מוצפנת רגילה מפענחת ומחזירה מחרוזת
hash_password - מקבלת סיסמה ומחזירה את הערך שלה לאחר גיבוב באמצעות אלגוריתם sha256 בצורה בטוחה

שם המחלקה Database:

תפקיד-

לטפל בכל הפעולות מול מסד הנתונים: יצירה, הוספה, שליפה, עדכון ומחיקה של טבלאות ונתונים עבור תלמידים, מורים, שיעורים, לוחות זמנים, אירועי התנהגות, נתוני התחברות וזיהוי פנים.

תכונות-

database - חיבור למסד הנתונים מסוג SQLite
cu - קורסור דרכו מתבצעות כל השאילתות
security - instance של מחלקת Security המשמש לגיבוב סיסמאות
face - instance של FaceAnalyzer שמבצע ניתוח תמונות פנים ומופעל ברמה גלובלית בקובץ

פעולות-

init - פעולה שמתחברת למסד הנתונים מהקובץ data.db ומוודאת שכל הטבלאות קיימות על ידי קריאה ל build_tables

build_tables - יוצרת את כל הטבלאות במסד אם הן לא קיימות כולל Students Teachers Classes
StudentClasses Schedule Behavior Logins וגם אינדקסים מתאימים

add_students - מוסיפה תלמידים חדשים עם טיפול בשגיאות של כפילויות
set_student_embedding - שומרת את הוקטור המספרי של פני תלמיד לתוך מסד הנתונים
get_student_classes - מחזירה את רשימת מזהי הכיתות של תלמיד מסוים
get_student_schedule - מחזירה את מערכת השעות של תלמיד כולל טיפול בשינויים זמניים שפג תוקפם
create_classes - מוסיפה כיתות חדשות לפי פרטי המורה והמקצוע עם טיפול בשגיאות כמו מורה שלא קיים
enroll_student_into_classes - משייכת תלמיד לכיתות לפי מזהים או לפי פרטי הכיתה
revert_expired_schedule_changes - בודקת האם קיימים שינויים זמניים שמועד הסיום שלהם עבר ומחזירה אותם לערכי ברירת מחדל
add_class_schedule - מוסיפה שיעור חדש למערכת השעות של כיתה
temporary_change - מאפשר לבצע שינוי זמני בשיעור מסוים כולל שינוי חדר ביטול או איפוס
register_face_embedding - מקבלת embedding של פנים ושומרת אותו בטבלה הרלוונטית לפי סוג המשתמש
add_teachers - מוסיף מורים למסד הנתונים עם בדיקה אם הם כבר קיימים
get_teacher_classes - מחזיר את הכיתות שמשייכות למורה
get_teacher_schedule - מחזיר את מערכת השעות של מורה כולל שינויים זמניים
get_students_from_class - מחזיר את מזהי כל התלמידים שמשייכים לכיתה מסוימת
log_behavior_events - שומר אירועים התנהגותיים לפי תעודת זהות וזמן
add_logins - מוסיף משתמשים למערכת ההתחברות עם סיסמה מוצפנת וסוג משתמש
find_device_token - בודק האם יש משתמש שקשור לטוקן של מכשיר ומחזיר את תעודת הזהות שלו
face_id_login - מבצע זיהוי פנים על ידי השוואת embedding שהתקבל מהתמונה מול כל המאגר ומחזיר את התוצאה אם נמצאה התאמה מספקת
verify_login_and_update_token - בודק האם תעודת זהות וסיסמה תקינים ואם כן מעדכן את הטוקן של המכשיר במסד הנתונים
close - סוגר את החיבור למסד הנתונים

שם המחלקה NetworkComponent:

תפקיד-

לשמור את פרטי ה- ip ו-port של השרת או הלקוח לפי הצורך
משמשת כבסיס להפעלת רשת במערכת

תכונות-

ip - כתובת ה- ip שדרכה מתבצעת התקשורת ברירת מחדל היא 127.0.0.1
port - מספר הפורט שבו מתבצעת התקשורת ברירת מחדל היא 8820

פעולות-

init- פעולה שמוגדרת עם יצירת האובייקט שומרת את הערכים של ip ו-port לשימוש בהמשך

קטעי קוד מיוחדים:

יכולת 1 – זיהוי פנים והשוואה -(face_id_login)

יכולת זו מאפשרת לתלמיד להתחבר למערכת על סמך תמונת פנים בלבד, ללא סיסמה. התמונה מומרת ל-embedding מספרי שמושואה לכל ה-embeddings הנתונים. אם נמצאת התאמה עם דמיון גבוה – המשתמש מזהה ונכנס.

```
def face_id_login(self, student_id, image_bytes):
    COSINE_THRESHOLD = 0.7
    EUCLIDEAN_THRESHOLD = 1.0

    # Step 1: Determine table
    self.cu.execute(
        """SELECT user_type FROM Logins WHERE id = ?", __parameters: (student_id,)
    row = self.cu.fetchone()
    if not row:
        return {"status": "fail", "output": "error type 1"}

    table = "Students" if row[0] == "student" else "Teachers"

    # Step 2: Load stored embedding
    self.cu.execute(
        """SELECT embedding FROM {table} WHERE id = ?", __parameters: (student_id,)
    row = self.cu.fetchone()
    if not row:
        return {"status": "fail", "output": "error type 1"}
    elif not row[0]:
        return {"status": "fail", "output": "error type 1"}
    elif len(row[0]) != 512 * 4:
        return {"status": "fail", "output": "error type 1"}

    stored_embedding = np.frombuffer(row[0], dtype=np.float32)

    # Step 3: Decode and write image
    image_data = base64.b64decode(image_bytes)
    temp_path = os.path.join("temp_face.jpg")
    with open(temp_path, "wb") as tmp:
        tmp.write(image_data)

    # Step 4: Get embedding from image
    captured_embedding = face.get_embedding(temp_path)
    os.remove(temp_path)

    # Validate captured embedding
    if not isinstance(captured_embedding, dict):
        return {"status": "fail", "output": "error type 2"}

    if "status" in captured_embedding and captured_embedding["status"] == "fail":
        return captured_embedding

    if "output" not in captured_embedding or not isinstance(captured_embedding["output"], list):
        return {"status": "fail", "output": "error type 2"}

    captured_embedding = np.array(captured_embedding["output"], dtype=np.float32)

    if captured_embedding.shape != (512,):
        return {"status": "fail", "output": "error type 2"}

    # Step 5: Normalize and compare
    stored_embedding = stored_embedding / np.linalg.norm(stored_embedding)
    captured_embedding = captured_embedding / np.linalg.norm(captured_embedding)

    def cosine_similarity(a, b):
        return np.dot(a, b)

    def euclidean_distance(a, b):
        return np.linalg.norm(a - b)

    cosine_sim = cosine_similarity(stored_embedding, captured_embedding)
    euclidean_dist = euclidean_distance(stored_embedding, captured_embedding)

    if cosine_sim > COSINE_THRESHOLD and euclidean_dist < EUCLIDEAN_THRESHOLD:
        return {"status": "success", "output": {"id": student_id, "type": table}}
    else:
        return {"status": "fail", "output": "error type 2"}
```

יכולת 2 ניהול הודעות בצד השרת מהלקוח (process_ message)

אהבתי את החלק הזה
בפרויקט שלי בגלל הדרך
שאני מטפל בכל
ההודעות בצורה דינמית
והיא לא תלויה בעוד
שניים/הוספות של
מחלקות.
בנוסף הדרך שאני מחזיר
תושבה עם python
dictionary, גם יפה
בעיניי, אפשר לדעת ישר
מה המצב "status" של
מה שהוחזר בלי להסתכל
או לבדוק את התוכן
ששמור ב"output"

```
def process_message(self, connection, data, fernet):
    db = Database.Database()
    face = FaceAnalyzer.FaceAnalyzer()
    try:
        message = self.s.decrypt_json(data, fernet)
        func_name = message["function"]
        args = message["args"] or {}

        if func_name == "capture_face_image":
            temp_path = face.capture_face_image()
            if temp_path == "fail":
                output = "fail"
            else:
                with open(temp_path, "rb") as f:
                    image_bytes = f.read()
                output = base64.b64encode(image_bytes).decode("utf-8")
                os.remove(temp_path)
        elif func_name == "get_back_icon":
            with open(os.path.join(BASE_DIR, "back_arrow.png"), "rb") as f:
                image_data = f.read()
            output = base64.b64encode(image_data).decode()
        elif func_name == "get_face_id_logo":
            with open(os.path.join(BASE_DIR, "face_id_logo.png"), "rb") as f:
                image_data = f.read()
            output = base64.b64encode(image_data).decode()
        elif func_name == "face_id_login":
            output = db.face_id_login(**args)
        else:
            func = getattr(db, func_name)
            output = func(**args)

        if isinstance(output, dict):
            response = output
        elif isinstance(output, list):
            response = {"status": "success", "output": output}
        elif output == "fail":
            if func_name == "verify_login_and_update_token":
                response = {"status": "fail", "output": "invalid login"}
            elif func_name == "capture_face_image":
                response = {"status": "fail", "output": "cannot get camera"}
            else:
                response = {"status": "error", "output": "internal server error"}
        else:
            response = {"status": "success", "output": output}

        response_bytes = self.s.encrypt_json(response, fernet)
        connection.sendall(response_bytes + b"<<END>>")
    except Exception as e:
        try:
            response = {"status": "error", "output": str(e)}
            connection.sendall(self.s.encrypt_json(response, fernet) + b"<<END>>")
        except:
            pass
```

בדיקות:

בדיקה התחברות עם סיסמה שגויה-

מטרת הבדיקה-

לודא שהמערכת לא מאשרת התחברות של משתמש שמזין סיסמה לא נכונה, גם אם מספר הזהות שלו נכון.

מה בוצע בפועל-

ניסיתי להתחבר עם תעודת זהות של תלמיד קיים והזנתי סיסמה שגויה בכוונה. הרצתי את הפעולה ממסך ההתחברות הרגיל, ולא שיניתי כלום בצד השרת או במסד הנתונים.

מה היה תוצאות הבדיקה-

המערכת זיהתה שהסיסמה לא תואמת לערך במסד הנתונים והחזירה את ההודעה "סיסמה שגויה". לא בוצעה התחברות, לא עודכן טוקן, ולא נשלפו פרטים מהמסד.

כיצד נפתרה הבעיה:

הבדיקה צלחה. לא היה צריך תיקון.

שינוי שיעור על ידי מורה שלא התעדכן מיד-

מטרת הבדיקה-

לבדוק האם לאחר שמורה משנה שיעור (למשל: מקום, שעה או ביטול), המידע מתעדכן מיד גם אצלו במסך.

מה בוצע בפועל-

ביצעתי שינוי זמני בשיעור דרך מסך שינוי השיעורים של מורה, לדוגמה העברת מחדר 111 ל 105. לאחר השמירה, יצאתי ממסך השינויים וחזרתי למסך הראשי.

מה היה תוצאות הבדיקה-

המערכת לא הציגה את השינוי שביצעתי. השיעור עדיין הופיע בגרסה הישנה שלו כאילו לא עשיתי כלום. היה צריך להיכנס ולצאת ידנית כדי לראות את המידע החדש.

כיצד נפתרה הבעיה:

לאחר שינוי, מחקתי את הפריים הראשי, וקראתי שוב לפונקציה המרכזית שמריצה את המסך הראשי, ככה כל העדכונים היו החדשים שבמסד.

עדכון תמונת פנים עם כמה אנשים בפריים-

מטרת הבדיקה-

לבדוק מה קורה אם בזמן צילום הפנים לרישום מופיעים כמה אנשים בפריים, ואם המערכת מתמודדת עם זה כמו שצריך.

מה בוצע בפועל-

ניסיתי לרשום embedding לפנים של תלמיד חדש, אבל בכוונה עמדו שני אנשים מול המצלמה יחד.

מה היה תוצאות הבדיקה-

הפונקציה קרסה והמערכת מכיוון שהיו כמה embeddings שמצאו.

כיצד נפתרה הבעיה:

הוספתי בדיקה שמוודאת שמופיעה רק פנים אחת בתמונה. אם יש יותר מפנים אחת או שאין בכלל – המערכת מחזירה שגיאה ולא מנסה להפיק embedding בכלל. ככה אין מצב ששומרים embedding שגוי או שמזוהה אדם לא נכון.

התחברות רגילה עם טוקן תקין ותמונת פנים נכונה-

מטרת הבדיקה-

לוודא שהמערכת מצליחה לצלם, לשלוח ולזהות תלמיד שנמצא מול המצלמה, כשלטוקן שלו שמור embedding מתואם במסד הנתונים.

מה בוצע בפועל-

הרשיתי לטוקן מסוים להתחבר אוטומטית והזנתי לו embedding מוקלט של התלמיד. אחר כך הרצתי את המערכת, חיכיתי לזיהוי אוטומטי, וישבתי מול המצלמה.

מה היה תוצאות הבדיקה-

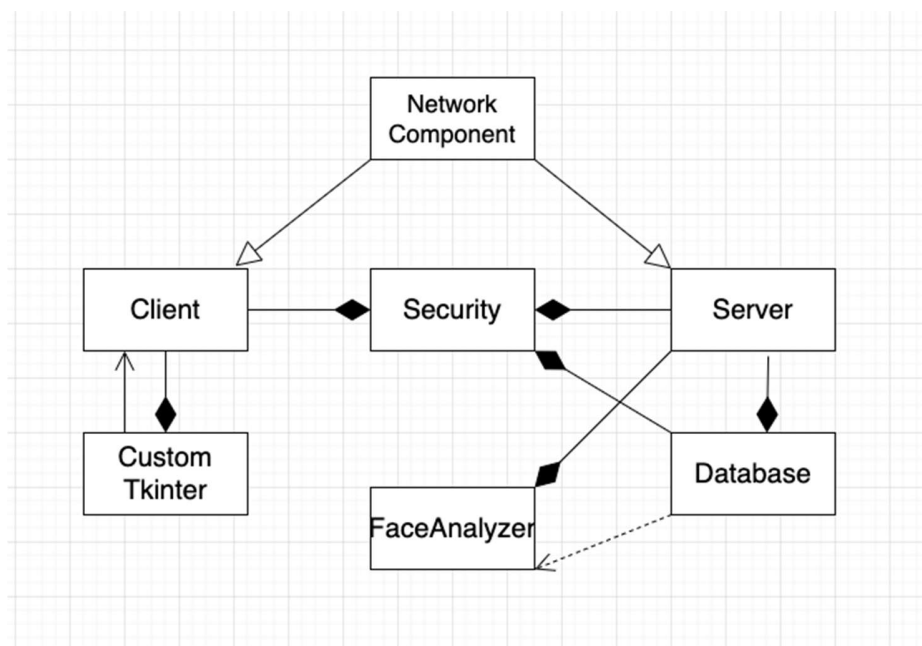
המערכת זיהתה אותי מיד, לא ביקשה סיסמה, ופתחה את המסך המרכזי של התלמיד בלי בעיות.

כיצד נפתרה הבעיה:

לא היו בעיות. הבדיקה עברה בהצלחה.

מדריך למשתמש

קבצים:



כפי שאפשר לראות רוב הקבצים נמצאים בצד השרת, והם-

-Security.py

קובץ של מחלקה שיצרתי, מטרתה להצפין את ההודעות בצורה דינמית.

-Database.py

קובץ מחלקה שיצרתי, מנהלת את מסד הנתונים

-FaceAnalyzer.py

קובץ אחראי על להכין את הספרייה האחראית על זיהוי פנים

-NetworkComponent.py

קובץ של מחלקה שיצרתי, ממנה יורשים סרבר וקליינט.

-private.pem

מפתח פרטי של הצפנה RSA.

-public.pem
מפתח ציבורי של הצפנה RSA, מועבר לקליינט בהתחברות.

-face_id_logo.png
קובץ תמונה של תהליך זיהוי פנים, הנמצא בהתחלה של ההתחברות.

-back_arrow.png
קובץ תמונה של חץ אחורה הנמצא ב-2 מהמסכים שלי

מתוך הקבצים האלה הקליינט עושה שימוש ב-Secuirty.py, NetworkComponent.py. ומחוץ לזה נדרש אצלו גם הקובץ

CustomTkinter.py - קובץ של מחלקה שיצרתי האחראי על ממשק הלקוח.

התקנה במערכת:

סביבת עבודה-

כדי שהמערכת תעבוד כמו שצריך, נדרש Python 3.11 ומעלה.
הקוד לא ירוץ בלי הספריות והמודלים המיובאים, לכן חובה להתקין את כולם לפני.

הכלים הנדרשים-

PyCharm - להרצה וכתיבה של הקוד

pip - חובה להתקנת הספריות

הספריות:
pip install customtkinter
pip install numpy
pip install cryptography
pip install onnxruntime
pip install insightface

כדי שהספרייה insightface תעבוד בלי שגיאות, חייבים להתקין גם את הכלים של הבילד:
pip install setuptools wheel

אם יש שגיאות הרצה שקשורות ל-C++ build tools:-
יש להתקין את Microsoft C++ Build Tools מאתר – Visual Studio חובה בשביל insightface

נתונים התחלתיים:

כדי לבדוק את המערכת ולוודא שהכול עובד כמו שצריך, הוזנו מראש נתונים לדוגמה אל תוך מסד הנתונים:

תלמידים:

- 1234 דניאל בנטוב
- 5678 תלמיד 2
- 2468 תלמיד 3

מורים:

- 9876 סטלה מורה
- 5432 בוריס
- 21 אהוד גורן
- 1010 דניס אנגליה

כיתות:

- י"ב 1 – מתמטיקה 5 יח"ל סטלה מורה
- י"ב 2 – סייבר רשתות / מדעי המחשב / פיזיקה בוריס / אהוד גורן
- י"ב 5 – סייבר פרויקטים בוריס
- י"ב 7 – אנגלית 5 יח"ל דניס אנגליה

מערכת שעות:

- מתמטיקה – יום שני שעות 2–4
- סייבר – יום שישי שעות 3–4
- פיזיקה – יום רביעי שעות 5–6
- מדעי המחשב – יום שני שעות 5–6
- סייבר פרויקטים – יום ראשון שעות 5–6
- אנגלית – יום ראשון שעה 5

התחברות:

קיימים חשבונות התחברות התחלתיים עם סיסמאות בדוקות מראש:

מורה:

משתמש 21 סיסמה 1234

תלמידים:

משתמש 1234, סיסמה 1234

משתמש 5678, סיסמה 3456

משתמש 2468, סיסמה 2345

רשת:

הרשת מבוססת על פרוטוקול TCP אשר מאפשר חיבור יציב בין מחשב השרת למחשב הלקוח. השרת מאזין לכתובת IP מסוימת ופורט פתוח, וכאשר לקוח מתחבר נוצר ערוץ תקשורת דרכו מועברים נתונים בצורה מוצפנת.

המערכת בנויה כך שכל לקוח פועל בתהליך נפרד ומקבל מענה אישי מהשרת. בתקשורת הזו עובר מידע כמו פרטי התחברות, זיהוי פנים, ובקשות לקבלת מערכת שעות.

הארכיטקטורה המינימלית הנדרשת להרצת המערכת כוללת:

מחשב אחד שישמש כשרת ומחשב נוסף כלקוח (או אחד שמריץ את שניהם דרך terminal)

מצלמה תקינה במחשב הלקוח (יעבוד בלי אבל צריך בשביל הזיהוי פנים)

אופן הפעלה המערכת:

בהרצת הקוד אוטומטית המערכת תחפש device token מתאים למכשיר שממנה רץ הקוד, טוקן כזה נשמר אחרי כניסה למשתמש באופן רגיל (תעודת זהות וסיסמה).

אם זאת פעם ראשונה שפותחים את האפליקציה, הטוקן לא נמצא ומגיעים למסך התחברות רגילה:

התחברות למשוב

תעודת זהות

סיסמה

show

התחבר

לכאן מכניסים את
תעודת זהות
וסיסמה של
המשתמש שאנחנו
רוצים להתחבר
אליו, לדוגמה:
תלמיד עם ת"ז:
1234, וסיסמה:
1234.

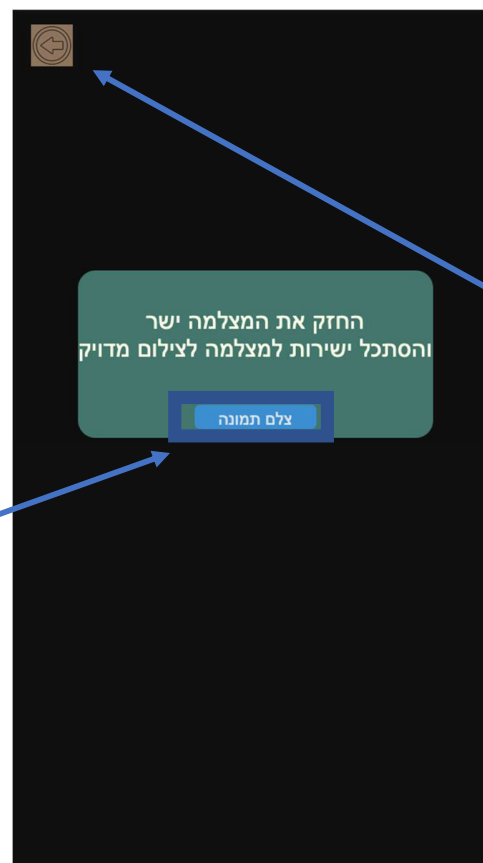
משם אם הזנו את הנתונים הנכונים, נגיע למסך המרכזי בפרויקט של המערכת שעות הנראה ככה:

העלה תמונת פנים		
ראשון	שני	שלישי
7:45 8:30		
8:30 9:15		
9:15 10:00	מתמטיקה 5 יח"ל	108 סטלה מורה
10:20 11:05	מתמטיקה 5 יח"ל	107 סטלה מורה
11:05 11:50	מתמטיקה 5 יח"ל	107 סטלה מורה
12:05 12:50	מדעי מחשב מבני נתונים	111 בוריס
12:50 13:55	מדעי מחשב מבני נתונים	111 בוריס
15:55		

כאן ניתן לראות את מערכת השעות של התלמיד, כולל כל השינויים הזמניים שהמורים שלו הזינו (ביטול או החלפת חדר)

ניתן לגלוז למטה לראות את שאר השעות ביום, או להחליף לימים האחרים בשבוע עם הכפתורים שמעל המערכת.

בלחיצה על כפתור "העלה תמונת פנים" יופיע מסך שבו יהיה ניתן יהיה להעלות תמונה למערכת, שתמיר אותה לembedding, ותשמור במסד נתונים.



לחיצה על כפתור החץ יחזיר אותנו למסך הראשי.

לחיצה על הכפתור במסך החדש, תצלם תמונה בעזרת המצלמה של המכשיר, ותחזיר תשובה לפי מה שהמצלמה תפסה.

במקרה של מצלמה לא תקינה, לא זוהה פנים, או שזוהו יותר מפנים אחת, יופיע הודעת שגיאה.

במקרה של תמונת פנים תקינה שעלתה, יופיע הודעת הצלחה, והembedding של אותה

פנים תעלה למסד נתונים, ליד הת"ז של אותו משתמש

נחזור להתחלה ונראה שכאשר אנחנו פותחים את האפליקציה מחדש מאותו מכשיר, עכשיו שהטוקן ותמונת פנים שמורות במסד, במקום המסך הראשון יופיע המסך הבא:

ככה אנחנו יודעים שהטוקן קיים במסד.

המערכת תיגש למצלמה, תצלם, ולפי הנתונים במסד יופיע המסך הבא.

במקרה והכל תקין, יופיע המסך הראשי שראינו מקודם, אך במקרה של פנים לא תואמות, יותר מפנים אחד בפריים, מצלמה לא תקינה או שלא קיים פנים שמורות במסד, יופיע לנו מסך שגיא עם 2 אפשרויות.



כל מה שראינו עד עכשיו תקף למורה בדיוק כמו שהוא תקף לתלמיד, אך יש שינוי אחד אצל מורים שאצל תלמידים אין וזה שינוי פרטי שיעור, כלומר בוטל או שהתחלף החדר.

המערכת אצל המורים נראת אותו דבר (חוץ מהשם שלהם בלמטה של כל שיעור), רק שאפשר ללחוץ על השיעורים עצמם כדי לגשת למסך הזה.

הכנס מספר כיתה חדש (רשות)

לדוגמה: 203

☐ בטל שיעור זה

בצע

איפוס שינויים

ניתן לשנות את מספר החדר שנמצאים בו, לבטל לגמרי את השיעור לפי ההנחיות על המסך.

מעבר לכך יש את הכפתור חץ כמו שראינו מקודם שנותן לנו לחזור למסך הראשי.

יש גם איפוס שינויים שיופיע רק כאשר יש שינוי לאפס, והוא יחזיר את הערכים כמו שהם אמורים להיות.

חוץ מאיפוס שינויים הכפתור, השינויים מתאפסים יום אחרי השיעור עצמו כדי לא ליצור בלבול עם השיעורים הבאים שבוע לאחר מכן.

רפלקציה:

כשהתחלתי את הפרויקט היו לי כל מיני רעיונות למה שאני ארצה שיהיה לי בפרויקט, רמת הקושי של כל דבר וכמה זמן כל חלק יקח לי. בפועל זה היה מאוד שונה, הרעיון המרכזי שהיה לי בהתחלה, שהוא סימון נוכחות עם זיהוי פנים לא יצא לפועל. חשבתי שכניסה עם זיהוי פנים רק יבוא בנוסף לכך אבל יצא בסוף שהוא החליף אותו, למרות שלא הספקתי עד הגשת הספר פרויקט לסיים את הרעיון, אני כן מתכנן להשלים אותו עד הבגרות עצמה.

מכמה סיבות, יצא שלא הייתי בבית ספר השנה ל-3-4 חודשים ולכן כמו בשאר המקצועות פספסתי הרבה בסייבר, בגלל זה הייתי צריך ללמוד בערך את הכל לבד בבית- בעיקר מסרטונים אתרים אחרי הכל למרות שהיה נדמה לי בהתחלה ששיעורי הבית ספר שאני אפספס רק יפגעו בי, לקחתי ולמדתי משהו שיעזור לי הרבה יותר בחיים וזה היכולת להשתמש בכל הכלים שיש לי כדי ללמוד לבד.

אני חושב שכל הכלים שניתנו לנו לשימוש הם מה שהייתי בוחר לעצמי, (או לפחות לפרויקט של ילד בתיכון) אבל אם הייתי חייב לשנות משהו זה היה להוריד מההנחיות ורק להציע לנו אותם כרשות. לדוגמה לא היה לי צורך בירושה בפרויקט, ולכן רק הוספתי אותו בסוף בשביל לסמן על ההנחיות "וי". בנוסף הייתי גם מרשה לתלמידים לפתח משחקים כמו שניתן לנו לעשות באסמבלי שנה קודם לכן, אני מבין שיותר רוצים שנבין את כל העקרונות של רשתות ופרויקטים אמיתיים, אבל לדעתי היינו יכולים ללמוד באותה מידה רק הרבה יותר בקלילות בפרויקט שבאמת רצינו לפתח.

אני רוצה להודות קודם כל למורה שהביא לי את הרעיון הראשוני, רום- למרות שהוא לא מלמד אותי הגיע אלי הרעיון שהוא תמיד רצה שתלמיד יעשה, שזה סימון נוכחות בשיעור עם זיהוי פנים של התלמידים, למרות שנכון לעכשיו זה לא יצא לפועל זה כן נתן לי כיוון לשאר הפרויקט. בנוסף אני רוצה להודות למורי סייבר שלי: עידן, עודי, ליאת וקובי. גם אם לא למדתי איתם בבית ספר רוב השנה הם תמיד היו פנויים לשאלות על כל נושא שיכל להיות בפרויקט.

לסיכום, ההשקעה שלי בפרויקט הייתה עמוקה ונמשכה לאורך זמן, ואני מרגיש שעברתי תהליך משמעותי באמת. מעבר ללמידה רכשתי גם הבנה רחבה יותר על איך מערכות כאלה עובדות. הפרויקט דרש ממני לחפש לבד פתרונות, לקרוא תיעוד, להתייעץ ולפעמים גם ללמוד טכנולוגיה חדשה מאפס. הופתעתי לגלות כמה אני מסוגל ללמוד לבד כשיש לי מטרה ברורה, וזה לקח חשוב שאני לוקח איתי להמשך.

ביבליוגרפיה:

1. קורס אינטרנטי של SQL, נגיש מ <https://www.w3schools.com/sql>.

2. פלייליסט סרטונים על CustomTkinter ביוטיוב. נגיש מ https://www.youtube.com/watch?v=Y01r643ckfl&list=PLfZw_tZWahxJl81b1S-vYQwHs_9ZT77f

2. למידה כללית עם OpenCv לכל הנושאים שקשורים לזיהוי פנים.

נסחפים:

רציתי להרחיב יותר על כל הנושא של הזיהוי פנים, זה נכון שהייתי יכול פשוט להעתיק קוד של ספרייה מוכנה לשימוש אבל זה הרגיש לי שאני מרמה קצת, לכן למדתי יותר לעומק על הנושא ואני רוצה להסביר אותו עכשיו.

בניגוד לאדם שמסתכל על תמונה ורואה פנים, מחשב לא רואה תמונה – הוא מתרגם את התמונה למערך מספרים (וקטור), ובפרט לוקטור בעל 512 רכיבים. כל רכיב כזה מייצג תכונה מסוימת בפנים – לדוגמה, אם זה היה רק דו-ממדי, אפשר לדמיין רכיב אחד שמייצג רוחב ורכיב אחר שמייצג גובה. ככל שמסויפים עוד רכיבים – התיאור של הפנים נעשה מדויק ומלא יותר.

ההשוואה בין שני וקטורים כאלה מתבצעת על ידי חישוב שני מדדים עיקריים: Euclidean distance ו-cosine similarity.

המרחק האוקלידי מודד את המרחק הישיר בין שני הווקטורים, כלומר "כמה הם רחוקים אחד מהשני".

הדמיון הקוסינוסי בודק את הזווית ביניהם – אם הם פונים פחות או יותר לאותו כיוון, נחשב שהם מייצגים את אותו אדם.

להלן החישובים של כל אחד מהם (זה יותר פשוט ממה שזה נראה סתם זה גדול)

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

$$\text{Euclidean Distance} = |X - Y| = \sqrt{\sum_{i=1}^{i=n} (x_i - y_i)^2}$$

X: Array or vector X

Y: Array or vector Y

x: Values of horizontal axis in the coordinate plane

y: Values of vertical axis in the coordinate plane

n: Number of observations



בשיטה שבה השתמשתי, אם הדמיון הקוסינוסי היה מעל ערך סף מסוים (למשל 0.7), והמרחק האוקלידי היה נמוך מערך מסוים (למשל 1.0), הוחלט שמדובר באותו אדם.

למדתי גם על שיטות לזיהוי פנים בתוך תמונה. קיימות שיטות ישנות כמו Haar Cascades שהן מהירות אבל לא מדויקות – הן משתמשות בחלון שמחליק על התמונה ומנסה לאתר תבנית של פנים. השיטה הזאת לא מתמודדת טוב עם תאורה שונה, פנים מוסתרות או זוויות שונות. לעומת זאת, שיטות חדשות מבוססות רשת נוירונים (DNN) כמו RetinaFace, מזהות פנים בצורה הרבה יותר מדויקת – הן מבצעות חיזוי של מיקום הפנים, מזהות נקודות עיגון (למשל עיניים ואף), ומתאימות את הפנים בדיוק.

לאחר הזיהוי, כל פנים מועברות למודל זיהוי כמו ArcFace, אשר ממיר את הפנים לווקטור בן 512 רכיבים. המודל הזה יודע לבדל בין אנשים שונים ולזהות כשמדובר באותו אדם גם אם זווית הפנים השתנתה או התאורה שונה.

במהלך הלמידה שלי גם חקרתי ספריות אחרות כמו face_recognition – זוהי ספרייה נפוצה ללמידת זיהוי פנים בפייתון, אך גיליתי שהיא הרבה פחות מדויקת מinsightface ולכן לא השתמשתי בה בפרויקט. המודלים של InsightFace עבדו טוב יותר, במיוחד מבחינת הפרדה בין אנשים דומים ודיוק בזיהוי.

CLIENT:

```
import socket
import CustomTkinter as UI
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import serialization, hashes
```



```
from cryptography.fernet import Fernet
import Security
import uuid
import platform
import hashlib
from NetworkComponent import NetworkComponent
from cryptography.hazmat.backends import default_backend

class Client(NetworkComponent):
    def __init__(self, ip="127.0.0.1", port=8820):
        super().__init__(ip, port)

        # instance of the security class to send and recv from server
        self.s = Security.Security()

        # Create the client socket
        self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        try:
            self.client.connect((ip, port))
            # 1. Receive public key from server
            buffer = b""
            while True:
                chunk = self.client.recv(4096)
                if not chunk:
                    break
                buffer += chunk
                if b"<<END>>" in buffer:
                    break

            pem_data = buffer.replace(b"<<END>>", b"")
            public_key = serialization.load_pem_public_key(pem_data,
backend=default_backend())

            # 2. Extract numbers from the public key
```

```
numbers = public_key.public_numbers()
e = numbers.e
n = numbers.n

# 3. Generate Fernet key
fernet_key = Fernet.generate_key() # 32 bytes

# 4. Convert Fernet key to integer
fernet_int = int.from_bytes(fernet_key, byteorder='big')

# 5. Encrypt Fernet key manually using RSA math
cipher_int = pow(fernet_int, e, n)

# 6. Convert encrypted int back to bytes to send
cipher_bytes = cipher_int.to_bytes((cipher_int.bit_length() + 7) // 8,
byteorder='big')

# 7. Send encrypted Fernet key to server
self.client.send(cipher_bytes)

# 5. Create Fernet object for later use
self.fernet = Fernet(fernet_key)

except ConnectionRefusedError:
    """when the server is off or isnt running, this is the usual problems
clients
face and can only be covered here on the clients side"""

    self.client = None
    self.fernet = None

# Create the UI and pass the client instance
self.ui = UI.CustomTkinter(self)

# get the 1.HARDWARE MAC ADDRESS +
```

```
# 2. OPERATING SYSTEM (WINDOWS, DARWIN, ANDROID...) +
# 3. MACHINE TYPE (x86_64, arm64...) using these 3 we ensure a
unique device token for all users.
device_token = f"{uuid.getnode()}-{platform.system()}-"
{platform.machine()}"

# encode the string to make it to bytes so we can hash it, then hexdigest
it to make it a readable 64 character string,
# this is done for security reasons mostly.
self.hash_device_token =
hashlib.sha256(device_token.encode()).hexdigest()
response = self.send_command(
    function="find_device_token",
    args={"device_token": self.hash_device_token}
)
if not response or response["status"] == "error":
    self.ui.running("offline")
elif response["status"] == "success":
    self.ui.running("face_id")
else:
    self.ui.running("normal")

def send_command(self, function, args=None):

    if not self.client or not self.fernet:
        return {"status": "error", "output": "server not available"}

    """ Sends a command to the server. """
    message = {
        "function": function,
        "args": args
    }
    encrypted = self.s.encrypt_json(message, self.fernet)
    self.client.sendall(encrypted + b"<<END>>")
    try:
```

```
chunks = []
timeout = 20 if function == "face_id_login" else 12
self.client.setTimeout(timeout)

buffer = b""
while True:
    try:
        chunk = self.client.recv(8192)
        if not chunk:
            break # connection closed
        buffer += chunk
        if b"<<END>>" in buffer:
            break
    except socket.timeout:
        break

response = buffer
self.client.setTimeout(None)

if b"<<END>>" not in response:
    return {"status": "error", "output": "incomplete response"}

response = response.replace(b"<<END>>", b"")

# 🐞 Debug: make sure it looks valid
if not response.startswith(b"gAAAA"):
    # Corrupted or partial Fernet token
    return {"status": "error", "output": "invalid encrypted data"}

decrypted = self.s.decrypt_json(response, self.fernet)

return decrypted

except socket.timeout:
    return {"status": "error", "output": "timeout"}
```

```
except BrokenPipeError:
    self.client.close()
    return {"status": "error", "output": "connection lost"}
except OSError:
    """Example scenario causing this:
    Server disconnects unexpectedly.
    Your socket closes in the listening thread.
    Meanwhile, you attempt a send from UI. → OSError is raised."""
    self.client.close()
    return {"status": "error", "output": "connection lost"}

def run(self):
    # Run the UI
    self.ui.running()

def close(self):
    """safely closes the client connection"""
    self.client.close()

if __name__ == "__main__":
    client = Client()
    client.run()
```

SERVER

```
import base64
import socket
import threading

from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.fernet import Fernet

import FaceAnalyzer
import Security
import Database

import os
BASE_DIR = os.path.dirname(__file__)

from NetworkComponent import NetworkComponent
class Server(NetworkComponent):
    def __init__(self, ip="127.0.0.1", port=8820):
        super().__init__(ip, port)

        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server.bind((self.ip, self.port))
        self.server.listen()

        # create an instance of the security class for more organised encryption
        and decryption
        self.s = Security.Security()

        # create an instance of the database class to get access and be the
        middle man
        # between the client and the database
        self.db = Database.Database()
```

```
def run(self):
    # prevent ddos flood attacks
    max_clients = 15
    self.clients = 0
    try:
        while True:
            if self.clients < max_clients:
                connection, address = self.server.accept()
                client_thread = threading.Thread(target=self.handle_client,
args=(connection,), daemon=True)
                client_thread.start()
                self.clients += 1
            except KeyboardInterrupt:
                pass
        finally:
            self.close()

def handle_client(self, connection):
    # create another instance of database since SQL doesn't allow multiple
    threads to use the same instance
    # (unless specified otherwise and that creates problems)
    db = Database.Database()
    face = FaceAnalyzer.FaceAnalyzer()
    with connection:
        # 1. Load private key
        BASE_DIR = os.path.dirname(__file__)
        try:
            with open(os.path.join(BASE_DIR, "private.pem"), "rb") as f:
                private_key = serialization.load_pem_private_key(f.read(),
password=None)
        except FileNotFoundError:
            connection.close()
    return
```

```
# 2. Extract public key from it
public_key = private_key.public_key()

# 3. Send public key to client
pem = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
connection.send(pem + b"<<END>>")

# 4. Receive encrypted Fernet key
encrypted_fernet_key = connection.recv(256) # 256 bytes for RSA-
2048

# 5. Decrypt Fernet key using private key
# 5. Convert encrypted key (bytes) to integer
cipher_int = int.from_bytes(encrypted_fernet_key, byteorder='big')

# 6. Extract private numbers
numbers = private_key.private_numbers()
d = numbers.d
n = numbers.public_numbers.n

# 7. Decrypt Fernet key with RSA math
fernet_int = pow(cipher_int, d, n)

# 8. Convert decrypted integer back to bytes
fernet_key = fernet_int.to_bytes((fernet_int.bit_length() + 7) // 8,
byteorder='big')

# 6. Create Fernet object
fernet = Fernet(fernet_key)

client = True
```



```
while client:
    buffer = b""
    while True:
        chunk = connection.recv(8192)
        if not chunk:
            client = False
            break
        buffer += chunk
        if b"<<END>>" in buffer:
            break

    data = buffer.replace(b"<<END>>", b"")
    self.process_message(connection, data, fernet)
    self.clients -= 1
    connection.close()

def process_message(self, connection, data, fernet):
    db = Database.Database()
    face = FaceAnalyzer.FaceAnalyzer()
    try:
        message = self.s.decrypt_json(data, fernet)
        func_name = message["function"]
        args = message["args"] or {}

        if func_name == "capture_face_image":
            temp_path = face.capture_face_image()
            if temp_path == "fail":
                output = "fail"
            else:
                with open(temp_path, "rb") as f:
                    image_bytes = f.read()
                    output = base64.b64encode(image_bytes).decode("utf-8")
                    os.remove(temp_path)
        elif func_name == "get_back_icon":
            with open(os.path.join(BASE_DIR, "back_arrow.png"), "rb") as f:
```

```
        image_data = f.read()
        output = base64.b64encode(image_data).decode()
    elif func_name == "get_face_id_logo":
        with open(os.path.join(BASE_DIR, "face_id_logo.png"), "rb") as f:
            image_data = f.read()
            output = base64.b64encode(image_data).decode()
    elif func_name == "face_id_login":
        output = db.face_id_login(**args)
    else:
        func = getattr(db, func_name)
        output = func(**args)

    if isinstance(output, dict):
        response = output
    elif isinstance(output, list):
        response = {"status": "success", "output": output}
    elif output == "fail":
        if func_name == "verify_login_and_update_token":
            response = {"status": "fail", "output": "invalid login"}
        elif func_name == "capture_face_image":
            response = {"status": "fail", "output": "cannot get camera"}
        else:
            response = {"status": "error", "output": "internal server error"}
    else:
        response = {"status": "success", "output": output}

    response_bytes = self.s.encrypt_json(response, fernet)
    connection.sendall(response_bytes + b"<<END>>")
except Exception as e:
    try:
        response = {"status": "error", "output": str(e)}
        connection.sendall(self.s.encrypt_json(response, fernet) +
b"<<END>>")
    except:
        pass
```

```
def close(self):  
    self.server.close()  
  
if __name__ == "__main__":  
    server = Server()  
    server.run()
```

Database

```
import datetime  
import os  
import sqlite3  
import numpy as np  
import Security  
import base64  
  
from FaceAnalyzer import FaceAnalyzer  
face = FaceAnalyzer() # global shared instance  
  
class Database:  
  
    def __init__(self):  
        # Connect to SQLite database
```

```
# check_same_thread=False → allows this connection to be used in  
threads (safe here because each thread gets its own instance)  
# timeout=5.0 → wait up to 5 seconds if the database file is locked before  
raising an error  
self.database =  
sqlite3.connect("/Volumes/HUN/פרויקט/mashov/database.db",  
check_same_thread=False, timeout=5.0)  
  
self.cu = self.database.cursor()  
self.cu.execute("PRAGMA foreign_keys = ON;") # for db browser  
self.security = Security.Security()  
self.build_tables()  
  
def build_tables(self):  
    self.cu.execute(f"""CREATE TABLE IF NOT EXISTS Students  
        (id TEXT PRIMARY KEY,  
        name TEXT,  
        embedding BLOB);""")  
    self.cu.execute("CREATE INDEX IF NOT EXISTS  
students_student_id_index ON Students(id)")  
  
    self.cu.execute(f"""CREATE TABLE IF NOT EXISTS Teachers  
        (id TEXT PRIMARY KEY,  
        name TEXT,  
        embedding BLOB);""")  
    self.cu.execute("CREATE INDEX IF NOT EXISTS  
Teachers_teacher_id_index ON Teachers(id)")  
  
    self.cu.execute(f"""CREATE TABLE IF NOT EXISTS Classes  
        (class_id INTEGER PRIMARY KEY AUTOINCREMENT,  
        grade TEXT,  
        number TEXT,  
        subject TEXT,  
        teacher_id TEXT,  
        FOREIGN KEY (teacher_id) REFERENCES Teachers(id) ON
```

```
DELETE CASCADE,
    UNIQUE (grade,number,subject));""")
self.cu.execute("CREATE INDEX IF NOT EXISTS class_id_index ON
Classes(class_id)")

self.cu.execute("""CREATE TABLE IF NOT EXISTS StudentClasses
(student_id TEXT,
class_id INTEGER,
FOREIGN KEY (student_id) REFERENCES Students(id) ON
DELETE CASCADE,
FOREIGN KEY (class_id) REFERENCES Classes(class_id) ON
DELETE CASCADE,
UNIQUE (student_id, class_id));""")
self.cu.execute("CREATE INDEX IF NOT EXISTS
student_classes_student_id_index ON StudentClasses(student_id)")
self.cu.execute("CREATE INDEX IF NOT EXISTS
student_classes_class_id_index ON StudentClasses(class_id)")

self.cu.execute("""CREATE TABLE IF NOT EXISTS Schedule
(class_id INTEGER,
day TEXT,
hour TEXT,
room TEXT,
status TEXT,
change_date TEXT DEFAULT NULL,
FOREIGN KEY (class_id) REFERENCES Classes(class_id)
ON DELETE CASCADE,
UNIQUE (class_id, day, hour))""")
self.cu.execute("CREATE INDEX IF NOT EXISTS
Schedule_class_id_id_index ON Schedule(class_id)")
self.cu.execute("CREATE INDEX IF NOT EXISTS
Schedule_day_class_id_hour_index ON Schedule(day, class_id, hour)")

self.cu.execute("""CREATE TABLE IF NOT EXISTS Behavior
(student_id TEXT,
```

```
        event TEXT,
        class_id INTEGER,
        hour TEXT,
        date TEXT,
        FOREIGN KEY (student_id) REFERENCES Students(id)
ON DELETE CASCADE,
        FOREIGN KEY (class_id) REFERENCES
Classes(class_id) ON DELETE CASCADE,
        UNIQUE (student_id, class_id, hour, date))""")
    self.cu.execute("CREATE INDEX IF NOT EXISTS Behavior_student_id
ON Behavior(student_id)")
    self.cu.execute("CREATE INDEX IF NOT EXISTS
Behavior_class_id_date_hour ON Behavior(class_id, date, hour)")

    self.cu.execute("""CREATE TABLE IF NOT EXISTS Logins (
        id TEXT,
        password_hash TEXT NOT NULL,
        user_type TEXT CHECK(user_type IN ('student', 'teacher')),
        device_token TEXT,
        expires_at TIMESTAMP,
        UNIQUE (id));

""")
    self.cu.execute("CREATE INDEX IF NOT EXISTS idx_logins_id ON
Logins(id)")
    self.cu.execute("CREATE INDEX IF NOT EXISTS idx_logins_user_type
ON Logins(user_type)")
    self.cu.execute("CREATE INDEX IF NOT EXISTS
idx_logins_device_token ON Logins(device_token)")

    self.database.commit()

# ----- #
# STUDENTS #
# ----- #
def add_students(self, students):
```

```
try:
    self.cu.executemany(f"INSERT INTO Students VALUES(?,?,?)",
students)
    self.database.commit()
    # user didn't make any mistakes
except sqlite3.IntegrityError:
    for student in students:
        try:
            self.cu.execute(f"INSERT INTO Students VALUES(?,?,?)",
student)
            self.database.commit()
        except sqlite3.IntegrityError as e:
            if "UNIQUE" in str(e):
                return(f"ERROR - DUPLICATE STUDENT - STUDENT ID:
{student[0]}") # ! !#
            elif "NOT NULL" in str(e):
                return(f"ERROR - MISSING DATA REQUIRED - STUDENT ID:
{student[0]}")
            else:
                return("ERROR - ", e)

def set_student_embedding(self, stu_id, embedding):
    # turns embedding into 32 bytes (instead of 64) numpy array and then
into bytes for storing as blob in database
    embedding_bytes = np.array(embedding, dtype=np.float32).tobytes()

    self.cu.execute("UPDATE Students SET embedding = ? WHERE id = ?",
        (embedding_bytes, stu_id))

    self.database.commit()

def get_student_classes(self, student_id):
    self.cu.execute("SELECT class_id FROM StudentClasses WHERE
StudentClasses.student_id = ?", (student_id,))
```

```
# makes a list of all the classes_ids #
return [row[0] for row in self.cu.fetchall()]

def get_student_schedule(self, student_id):
    classes_ids = self.get_student_classes(student_id)

    if not classes_ids:
        # "no classes found"
        return []

    # creates the ?,?,... string that goes in place of the classes ids to prevent
    # injection,
    # the length based on the amount of classes #
    placeholders = ", ".join("?" * len(classes_ids))
    query = f"""SELECT Classes.subject, Schedule.day, Schedule.hour,
        Schedule.room, Schedule.status, Schedule.change_date,
Teachers.name
        FROM Classes
        INNER JOIN Schedule ON Classes.class_id = Schedule.class_id
        INNER JOIN Teachers ON Classes.teacher_id = Teachers.id
        WHERE Classes.class_id IN ({placeholders})"""

    self.revert_expired_schedule_changes()

    self.cu.execute(query, classes_ids)
    class_schedules = self.cu.fetchall()

    self.database.commit()

    self.cu.execute(query, classes_ids)
    class_schedules = self.cu.fetchall()
    return class_schedules

# ----- #
# CLASSES #
# ----- #
```



```
def create_classes(self, classes):
    try:
        self.cu.executemany("INSERT INTO Classes (grade, number, subject,
teacher_id) VALUES(?, ?, ?, ?)",
            classes)
        self.database.commit()
    except sqlite3.IntegrityError:
        for class_ in classes:
            try:
                self.cu.execute("INSERT INTO Classes (grade, number, subject,
teacher_id) VALUES(?, ?, ?, ?)",
                    class_)
                self.database.commit()
            except sqlite3.IntegrityError as e:
                if "UNIQUE" in str(e):
                    return(f"ERROR - DUPLICATE CLASS (grade: {class_[1]},
number: {class_[2]}, subject: {class_[3]})")
                elif "NOT NULL" in str(e):
                    return(f"ERROR - MISSING DATA REQUIRED (grade:
{class_[1]}, number: {class_[2]}, subject: {class_[3]})")
                elif "FOREIGN KEY" in str(e):
                    return(f"ERROR - INVALID TEACHER ID (grade: {class_[1]},
number: {class_[2]}, subject: {class_[3]})")
                else:
                    return("ERROR - ", e)

# ----- #
# STUDENTS CLASSES #
# ----- #

def enroll_student_into_classes(self, student_id, classes=None,
class_ids=None):
    if classes is not None:
        class_ids = []
        for class_ in classes:
            self.cu.execute("SELECT class_id FROM Classes WHERE grade =
```

```
? AND number = ? AND subject = ?",
    (class_[0], class_[1], class_[2]))
    class_ids.append(self.cu.fetchone()[0])

    elif class_ids is None:
        return("ERROR - NO CLASS / CLASS_ID ENTERED")

    self.cu.executemany("INSERT OR IGNORE INTO StudentClasses
(student_id, class_id) VALUES(?, ?)",
        [(student_id, class_id) for class_id in class_ids])
    self.database.commit()

# ----- #
# CLASSES SCHEDULES #
# ----- #

def revert_expired_schedule_changes(self):
    today = datetime.datetime.today().date()
    self.cu.execute("""SELECT class_id, day, hour, room, status,
change_date FROM Schedule
        WHERE change_date IS NOT NULL""")
    rows = self.cu.fetchall()
    for row in rows:
        if row[5] and datetime.datetime.strptime(row[5], "%Y-%m-%d").date() <
today:
            original_room = row[3].split('±')[-1]
            self.cu.execute("""UPDATE Schedule SET room = ?, status = ?,
change_date = NULL
                WHERE class_id = ? AND day = ? AND hour = ?""",
                (original_room, "Active", row[0], row[1], row[2]))
            self.database.commit()

def add_class_schedule(self, grade=None, subject=None, number=None,
```

```
day=None, hour=None, room=None, status=None, class_id=None):
    if not class_id:
        self.cu.execute("SELECT class_id FROM Classes WHERE grade = ?
AND subject = ? AND number = ?",
                        (grade, subject, number))
        class_id = self.cu.fetchone()[0]
    if class_id:
        self.cu.execute("INSERT OR IGNORE INTO Schedule (class_id, day,
hour, room, status) VALUES(?, ?, ?, ?, ?)",
                        (class_id, day, hour, room, status))
        self.database.commit()

def temporary_change(self, class_id, day, hour, room, status, date, reset):
    if reset:
        self.cu.execute("SELECT room FROM Schedule WHERE class_id = ?
AND day = ? AND hour = ?",
                        (class_id, day, hour))
        current_room = self.cu.fetchone()[0]

        # Get the original room (first in the ± chain)
        original_room = current_room.split('±')[-1]

        # Overwrite the full field with the original room only
        self.cu.execute("""
            UPDATE Schedule
            SET room = ?, status = ?, change_date = NULL
            WHERE class_id = ? AND day = ? AND hour = ?
            """, (original_room, "Active", class_id, day, hour))
    else:
        self.cu.execute("SELECT room, status FROM Schedule WHERE
class_id = ? AND hour = ? AND day = ?",
                        (class_id, hour, day))
        change = self.cu.fetchone()

    if change:
```

```
if len(change[0]) > 100:
    return("ERROR - TOO MANY CHANGES") #! !#
if room != change[0]:
    self.cu.execute("UPDATE Schedule SET room = ?, change_date
= ? "
                    "WHERE class_id = ? AND day = ? AND hour = ?",
                    (str(room) + "±" + str(change[0]), date,
                     class_id, day, hour,))
if status != change[1]:
    self.cu.execute("UPDATE Schedule SET status = ? WHERE
class_id = ? AND day = ? AND hour = ?",
                    (status, class_id, day, hour))

self.database.commit()

def register_face_embedding(self, user_id, user_type, image_bytes):
    if isinstance(image_bytes, str):
        image_bytes = base64.b64decode(image_bytes)
        embedding_result = face.get_embedding(image_bytes)

        if embedding_result["status"] != "success":
            return {"status": "fail", "output": "ERROR - invalid face or no face
found"}

        embedding = embedding_result["output"]
        emb_bytes = np.array(embedding, dtype=np.float32).tobytes()

        if user_type == "student":
            self.cu.execute("UPDATE Students SET embedding = ? WHERE id =
?", (emb_bytes, user_id))
        elif user_type == "teacher":
            self.cu.execute("UPDATE Teachers SET embedding = ? WHERE id =
?", (emb_bytes, user_id))
```

```
else:
    return {"status": "fail"}

self.database.commit()
return {"status": "success"}

# ----- #
# TEACHERS #
# ----- #

def add_teachers(self, teachers):
    try:
        self.cu.executemany("INSERT INTO Teachers (id, name, embedding)
VALUES(?,?,?)",
                            teachers)
        self.database.commit()
    except sqlite3.IntegrityError:
        for teacher in teachers:
            try:
                self.cu.execute("INSERT INTO Teachers (id, name, embedding)
VALUES (?, ?, ?)",
                                teacher)
                self.database.commit()
            except sqlite3.IntegrityError as e:
                if "UNIQUE" in str(e):
                    return(f"ERROR - DUPLICATE TEACHER - TEACHER ID:
{teacher[0]}") # ! !#
                elif "NOT NULL" in str(e):
                    return(f"ERROR - MISSING DATA REQUIRED - TEACHER
ID: {teacher[0]}")
                else:
                    return("ERROR - ", e)

def get_teacher_classes(self, teacher_id):
```

```
self.cu.execute("SELECT class_id FROM Classes WHERE  
Classes.teacher_id = ?", (teacher_id,))  
# makes a list of all the classes_ids #  
row = self.cu.fetchall()  
return [result[0] for result in row]  
  
def get_teacher_schedule(self, teacher_id):  
    classes_ids = self.get_teacher_classes(teacher_id)  
  
    if not classes_ids:  
        return []  
  
    # creates the ?,?,... string that goes in place of the classes ids to prevent  
    # injection,  
    # the length based on the amount of classes #  
    placeholders = ", ".join("?" * len(classes_ids))  
    query = f"""SELECT Classes.subject, Schedule.day, Schedule.hour,  
        Schedule.room, Schedule.status, Schedule.change_date,  
Classes.grade,  
        Classes.class_id  
FROM Classes  
INNER JOIN Schedule ON Classes.class_id = Schedule.class_id  
INNER JOIN Teachers ON Classes.teacher_id = Teachers.id  
WHERE Classes.class_id IN ({placeholders})"""  
    self.cu.execute(query, classes_ids)  
    class_schedules = self.cu.fetchall()  
  
    self.revert_expired_schedule_changes()  
  
    self.cu.execute(query, classes_ids)  
    class_schedules = self.cu.fetchall()  
    return class_schedules  
  
# ----- #
```

```
# BEHAVIOR #
# ----- #

def get_students_from_class(self, class_id=None):
    self.cu.execute("SELECT student_id FROM StudentClasses WHERE
class_id = ?", (class_id,))
    self.database.commit()
    students = [first[0] for first in self.cu.fetchall()]
    return students

def log_behavior_events(self, students_events):
    # list of tuples (of everything (class_id, hour, date)) for students_events #
    self.cu.executemany("INSERT OR IGNORE INTO Behavior
(student_id, class_id, hour, date, event)VALUES (?, ?, ?, ?, ?)",
students_events)
    self.database.commit()

# ---- #
# LOGIN #
# ---- #

def add_logins(self, logins):

    try:
        self.cu.executemany("INSERT INTO Logins VALUES (?, ?, ?, ?, ?)",
logins)
        self.database.commit()
    except sqlite3.IntegrityError:
        for login in logins:
            try:
                self.cu.execute("INSERT INTO Logins VALUES (?, ?, ?, ?, ?)",
login)
                self.database.commit()
            except sqlite3.IntegrityError as e:
                if "UNIQUE" in str(e):
                    return(f"ERROR - DUPLICATE DEVICE TOKEN - ID:
```

```
{login[0]}")
    elif "NOT NULL" in str(e):
        return(f"ERROR - MISSING PASSWORD HASH - ID:
{login[0]}")
    elif "CHECK constraint failed" in str(e):
        return(f"ERROR - INVALID USER TYPE - ID: {login[0]}")
    else:
        return("ERROR -", e)

def find_device_token(self, device_token):
    self.cu.execute("SELECT id, user_type FROM Logins WHERE
device_token = ?", (device_token,))
    rows = self.cu.fetchall()

    if not rows:
        return {"status": "fail", "output": "error type 1"}

    return {"status": "success", "output": [{ "id": row[0], "type": row[1] } for row
in rows]}

def face_id_login(self, student_id, image_bytes):
    COSINE_THRESHOLD = 0.7
    EUCLIDEAN_THRESHOLD = 1.0

    # Step 1: Determine table
    self.cu.execute("SELECT user_type FROM Logins WHERE id = ?",
(student_id,))
    row = self.cu.fetchone()
    if not row:
        return {"status": "fail", "output": "error type 1"}

    table = "Students" if row[0] == "student" else "Teachers"

    # Step 2: Load stored embedding
    self.cu.execute(f"SELECT embedding FROM {table} WHERE id = ?",
```



```
(student_id,))
    row = self.cu.fetchone()
    if not row:
        return {"status": "fail", "output": "error type 1"}
    elif not row[0]:
        return {"status": "fail", "output": "error type 1"}
    elif len(row[0]) != 512 * 4:
        return {"status": "fail", "output": "error type 1"}

    stored_embedding = np.frombuffer(row[0], dtype=np.float32)

    # Step 3: Decode and write image
    image_data = base64.b64decode(image_bytes)
    temp_path = os.path.join("temp_face.jpg")
    with open(temp_path, "wb") as tmp:
        tmp.write(image_data)

    # Step 4: Get embedding from image
    captured_embedding = face.get_embedding(temp_path)
    os.remove(temp_path)

    # Validate captured embedding
    if not isinstance(captured_embedding, dict):
        return {"status": "fail", "output": "error type 2"}

    if "status" in captured_embedding and captured_embedding["status"] ==
"fail":
        return captured_embedding

    if "output" not in captured_embedding or not
isinstance(captured_embedding["output"], list):
        return {"status": "fail", "output": "error type 2"}

    captured_embedding = np.array(captured_embedding["output"],
dtype=np.float32)
```

```
if captured_embedding.shape != (512,):
    return {"status": "fail", "output": "error type 2"}

# Step 5: Normalize and compare
stored_embedding = stored_embedding /
np.linalg.norm(stored_embedding)
captured_embedding = captured_embedding /
np.linalg.norm(captured_embedding)

def cosine_similarity(a, b):
    return np.dot(a, b)

def euclidean_distance(a, b):
    return np.linalg.norm(a - b)

cosine_sim = cosine_similarity(stored_embedding, captured_embedding)
euclidean_dist = euclidean_distance(stored_embedding,
captured_embedding)

if cosine_sim > COSINE_THRESHOLD and euclidean_dist <
EUCLIDEAN_THRESHOLD:
    return {"status": "success", "output": {"id": student_id, "type": table}}
else:
    return {"status": "fail", "output": "error type 2"}

def verify_login_and_update_token(self, user_id: str, password: str, token):
    try:
        hashed = self.security.hash_password(password)
        # Starts a write transaction and locks the DB early to avoid conflicts
        during SELECT + UPDATE
        self.cu.execute("BEGIN IMMEDIATE")
        self.cu.execute("SELECT user_type FROM Logins WHERE id = ?
AND password_hash = ?", (user_id, hashed))
```

```
row = self.cu.fetchone()
if not row:
    # Cancels the transaction if login fails or an error occurs, releasing
    the DB lock cleanly
    self.database.rollback()
    return "fail"

    # Remove device token from any other account using it
    self.cu.execute("UPDATE Logins SET device_token = NULL WHERE
device_token = ? AND id != ?", (token, user_id))

    # assign it to the current user
    self.cu.execute("UPDATE Logins SET device_token = ? WHERE id =
?", (token, user_id))

    self.database.commit()
    return {"status": "success", "output": row}
except Exception as e:
    self.database.rollback()
    return {"status": "error", "output": str(e)}

# ---- #
# CLOSE #
# ---- #

def close(self):
    self.database.close()
```

Security:

```
import json
from cryptography.fernet import Fernet
import hashlib

class Security:
    # [data:str OR -> bytes] is a type hint, it's used for readability and so
    # that ide like pycharm can help and check for errors

    @staticmethod # static method since were not using self, this class is more
    utility
    # and is here just for quick and more readable code
    def encrypt_json(data, key: Fernet) -> bytes:
        return key.encrypt(json.dumps(data).encode())

    @staticmethod
    def encrypt(data: str, key: Fernet) -> bytes:
        return key.encrypt(data.encode())

    @staticmethod
    def decrypt_json(data: bytes, key: Fernet):
        return json.loads(key.decrypt(data).decode())

    @staticmethod
    def decrypt(data: bytes, key: Fernet) -> str:
        return key.decrypt(data).decode()

    @staticmethod
```

```
def hash_password(password: str) -> str:  
    return hashlib.sha256(password.encode()).hexdigest()
```

FaceAnalyzer:

```
import insightface  
import cv2  
import time  
import os  
import tempfile  
  
import numpy as np  
  
class FaceAnalyzer:  
  
    def __init__(self):  
        # 'buffalo_l' is a pretrained model that specialises in face detection and  
        recognition #  
        self.app = insightface.app.FaceAnalysis(name='buffalo_l')  
        # 0 indicates using a cpu rather than gpu as most phone's gpu won't be  
        sufficient for this procedure #  
        self.app.prepare(ctx_id=0)  
  
    def check_webcam_access(self):  
        try:  
            cap = cv2.VideoCapture(0)  
            if not cap.isOpened():  
                return False  
            cap.release() # <-- important!
```

```
        return True
    except Exception as e:
        return False

def get_embedding(self, image_input):
    if isinstance(image_input, str):
        # It's a path
        img = cv2.imread(image_input)

    elif isinstance(image_input, bytes):
        # Convert bytes → numpy array → OpenCV image
        nparr = np.frombuffer(image_input, np.uint8)
        img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    else:
        return {"status": "fail", "output": "unsupported input"}

    if img is None or img.mean() < 2 or img.shape[0] < 60 or img.shape[1] <
60:
        return {"status": "fail", "output": "error type 2"}

    faces = self.app.get(img)
    if not faces:
        return {"status": "fail", "output": "error type 2"}

    embedding = faces[0].embedding
    return {"status": "success", "output": embedding.tolist()}
```

CustomTkinter:

```
import datetime
import hashlib
import platform
import tempfile
import time
import uuid
import os
import base64
import customtkinter as ctk
import tkinter as tk
import cv2
from PIL import Image, ImageTk
from base64 import b64decode
from io import BytesIO

class CustomTkinter:
    def __init__(self, client):
        self.client = client

        self.SCREEN_WIDTH = 562
        self.SCREEN_HEIGHT = 1000

        self.root = ctk.CTk()

        self.root.geometry(f"{self.SCREEN_WIDTH}x{self.SCREEN_HEIGHT}")
        self.root.title("panima")

        self.logo_path = os.path.join(os.path.dirname(__file__),
"face_id_logo.png")
```

```
self.colors = ["#0D2B1D", "#345635", "#6B8F71", "#AEC3B0",  
"#E3EFD3"]  
self.schedule_background_color = "#254336"  
self.days_background_color = "#2a4d3e"  
self.kinda_blue = "#253745"  
self.dark_blue = "#11212D"  
self.very_light_blue = "#9BA8AB"  
self.text_color = "#F8FAE5"  
self.changed_room_color = "#d9c96f"  
self.column1_color = "#2c695b"  
self.column2_color = "#43766C"  
self.darker_brown_color = "#7E6752"  
self.button_bg_color = "#8f755e"  
self.accent_color = "#dbbfa4"  
self.gray_bg_cancelled_class = "#555555"  
self.gray_text_cancelled_class = "#AAAAAA"  
self.bright_brown = "#987655"  
self.bright_red = "#FA6B84"  
  
self.HEADER_HEIGHT = int(self.SCREEN_HEIGHT * 0.09)  
self.header_frame = ctk.CTkFrame(self.root,  
height=self.HEADER_HEIGHT, fg_color=self.days_background_color)  
self.header_frame.pack(fill="x", side="top", padx=5)  
  
self.day_to = "sunday"  
  
def capture_face_image(self, temp_path="temp_face.png"):  
    import os  
  
    if temp_path is None:
```



```
fd, temp_path = tempfile.mkstemp(suffix=".jpg")
os.close(fd)

try:
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        # ERROR - webcam not accessible
        return "fail"

    time.sleep(0.5)

    ret, frame = cap.read()
    cap.release()

    if not ret or frame is None or frame.mean() < 2 or frame.shape[0] < 60:
        # "ERROR - invalid frame captured"
        return "fail"

    cv2.imwrite(temp_path, frame)
    return temp_path
except Exception as e:
    return "fail"

def running(self, login_type="normal"):
    self.login_ui(login_type)
    self.root.mainloop()

def login_ui(self, login_type):

    if hasattr(self, "login_frame") and self.login_frame.winfo_exists():
        self.login_frame.destroy()

    self.login_frame = ctk.CTkFrame(self.root, fg_color=self.column2_color)
    self.login_frame.pack(fill="both", expand=True, padx=3, pady=3)

    self.error_label = ctk.CTkLabel(
```

```
self.login_frame,
text="",
text_color=self.bright_red,
fg_color=self.column2_color, # Match your app's background
bg_color=self.column2_color,
corner_radius=5,
font=("Arial", 32),
padx=10, # Enough to give padding without stretching too wide
pady=5,
)
self.error_label.place(relx=0.5, rely=0.7, anchor="center")
# self.error_label.configure(text="") # Keep it empty at first

# Title
title_label = ctk.CTkLabel(self.login_frame, text="למשוב התחברות",
font=("Arial", 42), text_color=self.text_color)
title_label.pack(pady=60)

if login_type == "offline":
    label = ctk.CTkLabel(self.login_frame, text="זמין לא שרת", font=("Arial",
30), text_color=self.bright_red)
    label.pack(pady=80)
    return

elif login_type == "normal":
    # ID Entry
    self.id_entry = ctk.CTkEntry(self.login_frame, placeholder_text="מספר
זהות תעודת", font=("Arial", 26))
    self.id_entry.pack(pady=10)

    # Password Entry
    self.password_entry = ctk.CTkEntry(self.login_frame,
placeholder_text="סיסמה", show="*", font=("Arial", 26))
    self.password_entry.pack(pady=10)
```

```
self.show_password = False # new variable to track state

def toggle_password_visibility():
    if self.show_password:
        self.password_entry.configure(show="*")
        toggle_btn.configure(text="hide", bg_color=self.bright_brown,
fg_color=self.bright_brown) # closed eye
        self.show_password = False
    else:
        self.password_entry.configure(show="")
        toggle_btn.configure(text="show", bg_color=self.bright_brown,
fg_color=self.bright_brown) # open eye
        self.show_password = True

toggle_btn = ctk.CTkButton(self.login_frame, text="show", width=30,
height=30, command=toggle_password_visibility)
toggle_btn.pack(pady=5)

# Login button
login_btn = ctk.CTkButton(self.login_frame, text="התחבר",
command=self.login_attempt,
fg_color=self.bright_brown)
login_btn.pack(pady=20)

else:
    # Load the face ID logo image and ensure it has an alpha channel
    (RGBA)
    logo_result = self.client.send_command("get_face_id_logo", args={})
    if logo_result["status"] == "success":
        image_data = b64decode(logo_result["output"])

        with open(self.logo_path, "wb") as f:
            f.write(image_data)
```

```
self.face_id_logo = Image.open(self.logo_path).convert("RGBA")

# Convert the image to a Tk-compatible format and store as attribute
to prevent garbage collection
resized = self.face_id_logo.resize((160, 160)) # width, height
self.tk_image = ImageTk.PhotoImage(resized)
# Create a regular Tkinter label to display the image inside the login
frame
self.face_image_label = tk.Label(self.login_frame,
image=self.tk_image, bg=self.column2_color)
# Add the label to the layout with vertical padding
self.face_image_label.pack(pady=100)

ctk.CTkLabel(self.login_frame, text="בתהליך פנים זיהוי", font=("Arial",
26),
text_color=self.text_color).pack(pady=1)

self.root.update() # Force Tkinter to render the UI now
temp_path = self.capture_face_image()

def try_again():
    self.login_frame.destroy()
    self.root.update_idletasks()
    self.login_ui("face_id")

if temp_path == "fail":
    self.error_label.configure(text="נסה מחוברת שהיא ודא. זמינה לא המצלמה")
    שוב.")

    ctk.CTkButton(self.login_frame, text="נסה שוב", command=try_again,
fg_color=self.button_bg_color).pack(pady=(90, 0))
    ctk.CTkButton(self.login_frame, text="רגילה כניסה",
command=lambda: self.running("normal"),
fg_color=self.bright_brown).pack(pady=5)

    return
```

```
with open(temp_path, "rb") as f:
    image_bytes = f.read()

image_b64 = base64.b64encode(image_bytes).decode("utf-8")

os.remove(temp_path)

# image_response["output"] should already be a base64-encoded
image sent from the server.

student_id_list = self.client.send_command(
    function="find_device_token",
    args={"device_token": self.client.hashed_device_token}
)["output"]

if not student_id_list:
    self.running("normal")
    return

student = student_id_list[0] # use the first match only

result = self.client.send_command(
    function="face_id_login",
    args={"student_id": student["id"], "image_bytes": image_b64}
)

if result["status"] == "success":
    self.type = result["output"]["type"]
    user_id = result["output"]["id"]

    if self.type == "Students":
        self.schedule(student_id=user_id)
    else:
```

```
self.schedule(teacher_id=user_id)
else:
    self.error_label.configure(text="שוב נסה, נכשל הפנים זיהוי")
    ctk.CTkButton(self.login_frame, text="שוב נסה", command=try_again,
                  fg_color=self.button_bg_color).pack(pady=(90, 0))
    ctk.CTkButton(self.login_frame, text="רגילה כניסה",
command=lambda: self.running("normal"),
                  fg_color=self.bright_brown).pack(pady=5)

def login_attempt(self):
    user_id = self.id_entry.get()
    password = self.password_entry.get()

    self.error_label.configure(text="")
    if not user_id:
        self.error_label.configure(text="זהות תעודת להזין נא")
        return

    if not password:
        self.error_label.configure(text="סיסמה להזין נא")
        return

    self.error_label.configure(text="")

    # get the 1.HARDWARE MAC ADDRESS +
    # 2. OPERATING SYSTEM (WINDOWS, DARWIN, ANDROID...) +
    # 3. MACHINE TYPE (x86_64, arm64...) using these 3 we ensure a
    unique device token for all users.
    device_token = f"{uuid.getnode()}-{platform.system()}-
{platform.machine()}"

    # encode the string to make it to bytes so we can hash it, then hexidigest
    it to make it a readable 64 character string,
    # this is done for security reasons mostly.
    hashed_device_token =
```

```
hashlib.sha256(device_token.encode()).hexdigest()

response = self.client.send_command(
    function="verify_login_and_update_token",
    args={"user_id": user_id, "password": password, "token" :
hashed_device_token}
)
if response["status"] == "success":
    self.type = response["output"][0] # teacher or student
    if self.type == "student":
        self.schedule(student_id=user_id) # pass the id properly!
    else:
        self.schedule(teacher_id=user_id)
    else:
        self.error_label.configure(text="שגויים נתונים")

def schedule(self, student_id=None, teacher_id=None):
    self.login_frame.destroy()

def upload_photo():
    overlay = ctk.CTkFrame(self.root, width=self.SCREEN_WIDTH,
height=self.SCREEN_HEIGHT, fg_color="#111111")
    overlay.place(relx=0.5, rely=0.5, anchor="center")
    overlay.lift()

result = self.client.send_command("get_back_icon", args={})

if result["status"] == "success":
    image_data = b64decode(result["output"])
    image_bytes = BytesIO(image_data)
    back_image = Image.open(image_bytes).resize((48, 48))
    back_icon = ImageTk.PhotoImage(back_image)
```

```
# Keep a reference so it doesn't get garbage collected
overlay.back_icon = back_icon

back_btn = ctk.CTkButton(
    overlay,
    image=back_icon,
    text="", # no text
    width=64,
    height=64,
    fg_color="transparent",
    hover_color="#333333",
    command=overlay.destroy
)
back_btn.place(x=15, y=15)

inner_frame = ctk.CTkFrame(overlay, width=480, height=500,
fg_color=self.column2_color, corner_radius=20)
inner_frame.place(relx=0.5, rely=0.4, anchor="center")

instruction_label = ctk.CTkLabel(
    inner_frame,
    text="מדויק לצילום למצלמה ישירות והסתכל\ישר המצלמה את החזק",
    font=("Arial", 28),
    text_color=self.text_color,
    justify="center"
)
instruction_label.pack(pady=40)

def capture_picture():
    user_id = student_id if student_id else teacher_id
    user_type = "student" if student_id else "teacher"
    temp_path = self.capture_face_image()
    if temp_path == "fail":
        ctk.CTkLabel(
            inner_frame,
```



```
        text="מאחד יותר שיש או ח\פנים אין, תקינה תמונה נמצאה לא: שגיאה",
        font=("Arial", 20),
        text_color=self.bright_red,
        fg_color=self.column2_color
    ).pack(pady=10)
    return

    with open(temp_path, "rb") as f:
        image_bytes = f.read()
    os.remove(temp_path)

    image_b64 = base64.b64encode(image_bytes).decode("utf-8")
    response =
self.client.send_command(function="register_face_embedding",
args={"user_id": user_id, "user_type": user_type, "image_bytes" : image_b64})
    if response["status"] != "success":
        ctk.CTkLabel(
            inner_frame,
            text="מאחד יותר שיש או ח\פנים אין, תקינה תמונה נמצאה לא: שגיאה",
            font=("Arial", 20),
            text_color=self.bright_red,
            fg_color=self.column2_color
        ).pack(pady=10)
    else:
        if response["status"] == "success":
            ctk.CTkLabel(
                inner_frame,
                text="בהצלחה נשמרה תמונה!",
                font=("Arial", 20),
                text_color=self.accent_color,
                fg_color=self.column2_color
            ).pack(pady=10)

    take_pic_btn = ctk.CTkButton(
```

```
        inner_frame,
        text="תמונה צלם",
        font=("Arial", 20),
        command=capture_picture
    )
    take_pic_btn.pack(pady=10)

upload_face_btn = ctk.CTkButton(
    self.header_frame,
    text="פנים תמונת העלה",
    fg_color=self.button_bg_color,
    text_color=self.text_color,
    corner_radius=10,
    height=40,
    width=160,
    command=upload_photo
)
upload_face_btn.place(relx=0.35, rely=0.5, anchor="w")

SCHEDULE_HEIGHT = int(self.SCREEN_HEIGHT * 0.8)

# a function to get the wrap length needed for different lengths of
subjects/names for better ui #
def get_wrap_length(text, font_size=28, font_type="Arial"):
    words = text.split(' ')
    font = tk.font.Font(family=font_type, size=font_size)

    if words and len(text) > 12:
        if len(words) == 1:
            return 250
        elif len(words) == 2:
            return font.measure(max(words[0], words[1], key=font.measure))
        elif len(words) == 4:
```

```
        return font.measure(words[0] + words[1] + " ")
    else:
        if len(words[0]) > len(words[1]) + len(words[2]):
            return font.measure(words[0])
        elif len(words[2]) > len(words[0]) + len(words[1]):
            return font.measure(words[0] + words[1] + " ")
        else:
            if len(words[0]) > len(words[2]):
                return font.measure(words[0])
            else:
                return font.measure(words[0] + words[1] + " ")
    else:
        return 350
```

```
start_hour = {"0": "7:45",
              "1": "8:30",
              "2": "9:15",
              "3": "10:20",
              "4": "11:05",
              "5": "12:05",
              "6": "12:50",
              "7": "15:55",
              "8": "14:45",
              "9": "13:35",
              "10": "16:20", }
end_hour = {"0": "8:30",
            "1": "9:15",
            "2": "10:00",
            "3": "11:05",
            "4": "11:50",
            "5": "12:50",
            "6": "13:55",
            "7": "14:40",
```

```
        "8": "15:30",
        "9": "16:20",
        "10": "17:05"}
# frames for each day of the week (excluding saturday)
days_frames = {}

    def create_day_frame(day, classes_in_day):
        days_frames[day] = ctk.CTkScrollableFrame(self.root,
height=SCHEDULE_HEIGHT, fg_color=self.schedule_background_color,
scrollbar_button_color=self.button_bg_color,
scrollbar_button_hover_color=self.button_bg_color)

        days_frames[day].columnconfigure((0, 1), weight=1) # weight = 1 to
allow it to expand if needed
        for i in range(11):
            class_ = next((classes for classes in classes_in_day if int(classes[2])
== i), None)

            days_frames[day].rowconfigure((i,), weight=1)
            column_1_frame = ctk.CTkFrame(days_frames[day],
fg_color=self.gray_bg_cancelled_class if class_
and class_[4] == "Cancelled"
else self.column1_color,
width=150, height=100)
            column_1_frame.grid(row=i, column=1, pady=3, sticky="e")
            column_1_frame.propagate(False)
            # create the 10 hours of the day #

            # get the class from that specific hour in the day. using next
because faster #
```

```
# check for cancelled classes #

# 1, 2, 3
hour_label = ctk.CTkLabel(column_1_frame, text=f'{i}',
                           font=('Roboto', 36),
                           text_color=self.text_color)
hour_label.place(relx=0.85, rely=0.5, anchor="e")

# 7:45, 8:30
timeframe_label = ctk.CTkLabel(column_1_frame,
                                text=f'{start_hour[f"{i}"]} \n \n {end_hour[f"{i}"]}',
                                text_color=self.text_color,
                                font=("Roboto", 20))
timeframe_label.place(anchor="w", relx=0.1, rely=0.5)

# line between the times
line_width = 50 if i > 1 else 40
line_between_timeframe_label = ctk.CTkFrame(column_1_frame,
                                              height=3, width=line_width,
                                              fg_color=self.accent_color)
line_between_timeframe_label.place(rely=0.5, relx=0.1)

column_2_frame = ctk.CTkFrame(days_frames[day],
                               width=412, height=100,
                               fg_color=self.gray_bg_cancelled_class if class_
and class_[4] == "Cancelled" else self.column2_color)
column_2_frame.grid(row=i, column=0, sticky="w", pady=3, padx=3)
column_2_frame.propagate(False)

def open_edit_overlay(class_info, day, hour):
    overlay = ctk.CTkFrame(
        self.root,
        width=self.SCREEN_WIDTH,
        height=self.SCREEN_HEIGHT,
```

```
        fg_color=self.column2_color # or
self.schedule_background_color
    )
    overlay.place(relx=0.5, rely=0.5, anchor="center")
    overlay.lift()

    # Back arrow
    result = self.client.send_command("get_back_icon", args={})
    if result["status"] == "success":
        image_data = b64decode(result["output"])
        back_image = Image.open(BytesIO(image_data)).resize((48,
48))

        back_icon = ImageTk.PhotoImage(back_image)
        overlay.back_icon = back_icon # prevent garbage collection

        back_btn = ctk.CTkButton(
            overlay, image=back_icon, text="",
            width=64, height=64,
            fg_color="transparent",
            hover_color="#333333",
            command=overlay.destroy
        )
        back_btn.place(x=15, y=15)

    room_label = ctk.CTkLabel(
        overlay,
        text="חדש כיתה מספר הכנס (רשות)",
        font=("Arial", 22),
        text_color=self.text_color
    )
    room_label.place(relx=0.5, rely=0.32, anchor="center")

    room_entry = ctk.CTkEntry(
        overlay,
```

```
placeholder_text="לדוגמה: 203",
font=("Arial", 22),
height=35,
corner_radius=10
)

room_entry.place(relx=0.5, rely=0.4, anchor="center",
relwidth=0.4)

cancel_var = tk.BooleanVar()
cancel_checkbox = ctk.CTkCheckBox(
    overlay,
    text="זה שיעור בטל",
    variable=cancel_var,
    font=("Arial", 28),
    text_color=self.text_color,
    checkbox_height=32,
    checkbox_width=32,
    border_width=2,
    fg_color=self.accent_color,
    hover_color=self.bright_brown,
    border_color=self.text_color,
)

cancel_checkbox.place(relx=0.5, rely=0.55, anchor="center")

def on_reset():
    class_id = class_info[7]
    self.client.send_command("temporary_change", {
        "class_id": class_id,
        "day": day,
        "hour": hour,
        "room": "", # doesn't matter, will be ignored if reset=True
        "status": "", # same
        "date": "", # same
```

```
        "reset": True
    })

    overlay.destroy()
    for widget in self.root.winfo_children():
        if widget != self.header_frame:
            widget.destroy()
    self.schedule(teacher_id=teacher_id)

    if "±" in class_info[3] or class_info[4] == "Cancelled":
        reset_btn = ctk.CTkButton(
            overlay,
            text="שינויים איפוס",
            font=("Arial", 22),
            command=on_reset,
            height=48,
            fg_color=self.button_bg_color
        )
        reset_btn.place(relx=0.5, rely=0.9, anchor="center",
relwidth=0.6)

    def on_done():
        room_text = room_entry.get()
        status = "Cancelled" if cancel_var.get() else "Active"
        class_id = class_info[7] # you'll need to ensure this is included
in class_info
        today = datetime.datetime.today().strftime('%Y-%m-%d')

        self.client.send_command("temporary_change", {
            "class_id": class_id,
            "day": day,
            "hour": hour,
            "room": room_text if room_text else class_info[3],
            "status": status,
            "date": today,
```



```
        "reset": False
    })

    overlay.destroy()
    for widget in self.root.winfo_children():
        if widget not in [self.header_frame]:
            widget.destroy()
    self.schedule(teacher_id=teacher_id)

done_btn = ctk.CTkButton(
    overlay,
    text="בצע",
    font=("Arial", 26),
    command=on_done,
    height=60,
    fg_color=self.bright_brown
)
done_btn.place(relx=0.5, rely=0.8, anchor="center", relwidth=0.6)

# class exists on certain hour and day #
if class_:

    if teacher_id:
        column_2_frame.destroy()
        column_2_frame = ctk.CTkButton(
            days_frames[day],
            width=412,
            height=100,
            text="",
            fg_color=self.gray_bg_cancelled_class if class_[
                4] == "Cancelled" else
self.column2_color,
            hover_color=self.days_background_color,
            command=lambda c=class_, d=day, h=i:
```

```
open_edit_overlay(c, d, h)

    )
    column_2_frame.grid(row=i, column=0, sticky="w", pady=3,
padx=3)
    column_2_frame.propagate(False)
    subject_label = ctk.CTkLabel(column_2_frame,
                                font=("Arial", 28),
                                wraplength=get_wrap_length(class_[0]),
                                text=class_[0],
                                text_color=self.gray_text_cancelled_class if
class_[4] == "Cancelled"
                                else self.text_color,
                                fg_color=self.gray_bg_cancelled_class if
class_[4] == "Cancelled" else self.column2_color)
    subject_label.place(anchor="center", relx=0.8, rely=0.5)

    classroom_label = ctk.CTkLabel(column_2_frame,
                                   text=class_[3].split("±")[0],
                                   text_color=self.gray_text_cancelled_class if
class_[4] == "Cancelled"
                                   else self.changed_room_color if
len(class_[3].split('±')) > 1 else self.text_color,
                                   font=("Arial", 28))
    classroom_label.place(anchor="center", rely=0.4, relx=0.2)

    teacher_name_label = ctk.CTkLabel(column_2_frame,
                                       text=class_[6],
                                       text_color=self.gray_text_cancelled_class if
class_[4] == "Cancelled" else self.text_color,
                                       font=("Arial", 18),
                                       wraplength=get_wrap_length(class_[6]))
    teacher_name_label.place(anchor="center", rely=0.75, relx=0.2)
```

```
numbered_days = {
    1: "sunday",
    2: "monday",
    3: "tuesday",
    4: "wednesday",
    5: "thursday",
    6: "friday"
}

if student_id:
    all_hours_all_days =
self.client.send_command(function="get_student_schedule",
                           args={"student_id": student_id})["output"]

elif teacher_id:
    all_hours_all_days =
self.client.send_command(function="get_teacher_schedule",
                           args={"teacher_id": teacher_id})["output"]

# create all the frames for all the days #

for i in range(1, 7):
    # get all the classes of a certain day from all the classes in all the days
#
    classes_in_day_ = [
        class_ for class_ in all_hours_all_days
        if len(class_) > 1 and class_[1].lower() == numbered_days[i].lower()
    ]
    create_day_frame(numbered_days[i], classes_in_day_)

def switch_day(day_from, day_to_function):

    self.day_to = day_to_function
    # forget the frame currently on screen #
    days_frames[day_from].pack_forget()
```

```
# showcase the day that's been clicked #
days_frames[self.day_to].pack(side="bottom", fill="x", padx=5)

DAYS_HEIGHT = int(self.SCREEN_HEIGHT * 0.1)
all_days_frame = ctk.CTkFrame(self.root, height=DAYS_HEIGHT,
fg_color=self.days_background_color)
all_days_frame.pack(fill="x", side="top", padx=5, pady=3)

days_buttons = {}
hebrew_days = {
    "sunday": "ראשון",
    "monday": "שני",
    "tuesday": "שלישי",
    "wednesday": "רביעי",
    "thursday": "חמישי",
    "friday": "שישי"
}
num_of_days = len(numbered_days)
for i, day in enumerate(numbered_days.values()):
    days_buttons[day] = ctk.CTkButton(all_days_frame,
                                     text=hebrew_days[day],
                                     font=("Arial", 23),
                                     command=lambda d=day:
switch_day(self.day_to, d),
                                     fg_color=self.button_bg_color,
                                     border_color=self.darker_brown_color,
                                     border_width=2,
                                     hover_color=self.days_background_color,
                                     width=89,
                                     corner_radius=5,
                                     height=int(0.7 * DAYS_HEIGHT))
    days_buttons[day].place(anchor="center", rely=0.5, relx=(0.915 - 0.166
* i))
    days_buttons[day].propagate(False)
```

```
# rel x explanation - take away each time so the days are shown RTL  
instead of LTR. start at 0.915 so  
# the first one has space, and take away 0.166*i  
# so that the days are evenly spread out#  
  
# current day #  
today = datetime.datetime.today().weekday()  
  
# account for the way python handles days and their number, monday -  
0, tuesday - 1... #  
switch_day(self.day_to, numbered_days[1] if today > 4 else  
numbered_days[today+2])  
  
def update_class_ui(self,):  
    for widget in self.root.winfo_children():  
        if widget != self.header_frame:  
            widget.destroy()  
  
    self.schedule(student_id=self.student_id if hasattr(self, "student_id") else  
None,  
                 teacher_id=self.teacher_id if hasattr(self, "teacher_id") else  
None)
```

NetworkComponent:

```
class NetworkComponent:  
    def __init__(self, ip="127.0.0.1", port=8820):  
        self.ip = ip  
        self.port = port
```

Main:

```
import time

import Database
import numpy as np
import datetime
import Server
import Client
import threading
import Security as s
from FaceAnalyzer import FaceAnalyzer

#####

db = Database.Database()
db.__init__()
sec = s.Security

analyzer = FaceAnalyzer()

embedding = analyzer.get_embedding("random_dude.jpg")["output"]

if embedding is None:
    raise ValueError("Failed to get embedding")
a_bytes = np.array(embedding, dtype=np.float32).tobytes()
b_bytes = np.array("b").tobytes()
c_bytes = np.array("c").tobytes()
students = [("1234", "בנטוב דניאל", a_bytes), ("5678", "2 תלמיד", b_bytes),
            ("2468", "3 תלמיד", c_bytes)]

teachers = [("9876", "מורה סטלה", b_bytes), ("5432", "בוריס", c_bytes),
            ("21", "גורן אהוד", c_bytes), ("1010", "אנגליה דניס", b_bytes)]
```

```
classes = [("יב", "1", "יח"ל 5 מתמטיקה", "9876"), ("יב", "2", "רשתות סייבר", "5432"),  
           ("יב", "2", "פיזיקה", "21"), ("יב", "2", "מבני מחשב מדעי", "5432"),  
           ("יב", "5", "פרוייקטים סייבר", "5432"), ("יב", "7", "יח"ל 5 אנגלית", "1010")]  
  
db.add_students(students)  
db.add_teachers(teachers)  
  
db.create_classes(classes)  
  
db.enroll_student_into_classes("1234", class_ids=("1", "2", "3", "4", "6"))  
db.enroll_student_into_classes("5678", class_ids=("1", "3", "6"))  
db.enroll_student_into_classes("2468", class_ids=("1", "2", "5"))  
  
db.add_class_schedule(class_id=1, day="monday", hour="2", room=107,  
status="Active")  
db.add_class_schedule(class_id=1, day="monday", hour="3", room=107,  
status="Active")  
db.add_class_schedule(class_id=1, day="monday", hour="4", room=107,  
status="Active")  
  
db.add_class_schedule(class_id=2, day="friday", hour="3", room=227,  
status="Active")  
db.add_class_schedule(class_id=2, day="friday", hour="4", room=227,  
status="Active")  
  
db.add_class_schedule(class_id=3, day="wednesday", hour="5", room=111,  
status="Active")  
db.add_class_schedule(class_id=3, day="wednesday", hour="6", room=111,  
status="Active")  
  
db.add_class_schedule(class_id=4, day="monday", hour="5", room=111,  
status="Active")  
db.add_class_schedule(class_id=4, day="monday", hour="6", room=111,
```

```
status="Active")

db.add_class_schedule(class_id=5, day="sunday", hour="5", room=111,
status="Active")
db.add_class_schedule(class_id=5, day="sunday", hour="6", room=111,
status="Active")

db.add_class_schedule(class_id=6, day="sunday", hour="5", room=111,
status="Active")

db.temporary_change(1, "monday", 3, 207, "Active",
datetime.datetime.today().strftime('%Y-%m-%d'))
db.temporary_change(1, "monday", 4, 207, "Cancelled",
datetime.datetime.today().strftime('%Y-%m-%d'))

now = datetime.datetime.now()
future = now + datetime.timedelta(days=30)

logins = [("21", sec.hash_password("1234"), "teacher", "1", future), ("1234",
sec.hash_password("1234"), "student", "1", future), ("5678",
sec.hash_password("3456"), "student", "2", future),
          ("2468", sec.hash_password("2345"), "student", "3", future)]
db.add_logins(logins)

.....

def run_server():
    try:
        server = Server.Server()
        server.run()
```



```
except Exception as e:
    print("SERVER THREAD ERROR:", e)

def run_client():
    client = Client.Client()
    client.run()

if __name__ == "__main__":
    server_thread = threading.Thread(target=run_server)
    server_thread.start()

    time.sleep(1) # Give server a moment

    run_client() # UI must run in main thread

#####
db.close()
print('closed db')
#####
```